

THE COMPLETE PROJECT HANDBOOK

FinResearch AI

Understand the project.
Trace the code.
Defend the design.

A detailed, beginner-friendly guide to the Multi-Agent Financial Research System — from first principles to implementation limits and production improvements.

Includes every tracked file, the full database field dictionary, six agent walkthroughs and 60 interview/viva questions.

Repository

Gauravcodesbyheart/

multi-agent-financial-research-research-system

SOURCE-VERIFIED / BEGINNER TO ADVANCED

Architecture • Agents • Setup • Data • APIs • Project defense

Reviewed 20 September 2026 | Snapshot 482d9ae1d376

Contents

Click a chapter title or use your PDF viewer's bookmarks to navigate. Appendix A is the field-by-field schema reference; Appendix B maps all 69 tracked files.

1. How to read this handbook	5
2. The project explained from zero	6
3. Web-development foundations you need	7
4. Architecture and end-to-end data flow	9
5. Tools and technologies: what each one does	11
6. Set up the project from scratch	13
7. Beginner walkthrough and a safe demonstration	16
8. Document Agent and file ingestion in detail	18
9. Retrieval and RAG: what is really implemented	20
10. Extraction Agent and financial normalization	22
11. Financial concepts and every metric family	24
12. Risk Agent: signals, evidence and limitations	26
13. Research Agent: context, answers and citations	27
14. Benchmark Agent and Report Agent	28
15. Gemini integration, modes and cost model	30
16. Database design, relationships and persistence	32
17. API reference and request contracts	34
18. Frontend pages, state and formatting	37

19. Authentication, authorization and security review	39
20. Reliability, performance and consistency	41
21. What was tested and how to test properly	42
22. Troubleshooting guide	44
23. Deployment and operations without overclaiming	46
24. Implementation reality versus common project claims	47
25. Roadmap: improve the project in a sensible order	48
26. Presentation scripts and design-defense answers	49
27. Interview and viva question bank	51
28. Glossary and quick revision sheet	55
29. Maintaining and regenerating this handbook	56
Appendix A. Complete database field dictionary	57
Appendix B. Every tracked file and where to start	67
Appendix C. Agent function and data-contract map	74
Appendix D. Source evidence and review navigation	76

About this edition

A beginner-to-advanced, code-grounded guide to the Multi-Agent Financial Research System.

Repository: [Gauravcodesbyheart/multi-agent-financial-research-research-system](https://github.com/Gauravcodesbyheart/multi-agent-financial-research-research-system). Reviewed snapshot: 482d9ae1d3765ef7d5494348b1e17ae1286a68d2. Review date: 20 September 2026. Prepared from the repository's application code, schema, migration, configuration, dependency manifests, seed data and existing documentation.

This is an educational engineering handbook, not investment advice or a certification of production readiness. Examples marked illustrative are teaching examples, not verified market data. Findings describe this snapshot; future fixes may change behavior.

1. How to read this handbook

Three routes through the material

If you are completely new, read Chapters 2–5 first. They explain what the project is, what a web application does, and why each part exists. Then follow the setup and demonstration in Chapters 6–7 before reading the agent internals. Do not try to memorize every database column on your first reading.

If you are preparing for a presentation or viva, read the architecture, six-agent walkthrough, implementation-versus-claims discussion, security review and question bank. Practice tracing one upload and one chat question through the source files. Being able to explain a real function is more convincing than repeating the phrase “multi-agent AI.”

If you are maintaining the code, use the API reference, schema dictionary, file inventory, failure analysis and improvement roadmap. The appendices enumerate the actual source files and database fields, so this document also works as a lookup reference.

Evidence labels used throughout

Implemented means the behavior is visible in the reviewed code. **Limitation** means the current code is incomplete, fragile, inconsistent or bounded. **Recommendation** describes a possible change, not something the project already does. **Verified check** means a command was executed during this review. A source inspection is not the same as a successful real-world integration test.

Scope and verification boundaries

All 50 files under src were inspected for their purpose, route behavior, data access, agent logic or UI flow. The review also examined the 11-table migration, schema metadata, package manifests, application configuration, existing documentation and PDF-generation script. Dependency internals and every line of package-lock.json were not audited as application source.

The environment had no configured project database or Gemini credentials. Therefore login against a live database, upload processing against PostgreSQL, real Gemini responses and multi-user attacks were not exercised end to end. TypeScript, ESLint and a production build were run after installing dependencies using a documented workaround. Their exact outcomes appear in Chapter 21.

This handbook intentionally corrects older claims rather than copying them. In particular: six agent modules exist, keyword retrieval is implemented, the report download is TXT, and successful completion does not prove financial accuracy.

2. The project explained from zero

What problem is being solved?

A financial analyst often needs to read long company reports, locate revenue and profit figures, inspect risks, compare competitors and write a summary. Doing this manually means switching between PDFs, spreadsheets, notes and a report editor. Information may be scattered across pages and repeated in different units or periods.

FinResearch AI puts these activities into one browser-based workspace. A user can create an account, organize research into sessions, upload documents, inspect extracted values and risks, compare companies, ask questions and generate a report. Its audience includes students, MBA candidates, researchers and early-career analysts.

A useful everyday analogy

Imagine an analyst's office. The receptionist verifies who you are: authentication. A filing cabinet stores reports and research folders: PostgreSQL. A clerk divides long reports into manageable cards: the Document Agent. An accountant copies key figures: the Extraction Agent. A reviewer highlights potential dangers: the Risk Agent. A comparison analyst compares competitors: the Benchmark Agent. A research assistant answers your questions: the Research Agent. A writer compiles the report: the Report Agent.

In the actual code, these are TypeScript functions, not six humans and not six independent AI services. Five modules can call Gemini; document chunking itself is deterministic code. The application decides when each function runs.

What goes in and what comes out?

Inputs: user account details, research-session names and tags, uploaded PDF/DOCX/TXT files, optional company and fiscal-year metadata, selected companies, chat questions and optional report instructions.

Stored intermediate data: extracted document text, overlapping text chunks, financial-metric rows, risk flags, chat history, reports and activity logs.

Outputs: dashboards and charts, document details, source excerpts, conversational answers, comparison narratives, structured report metadata and downloadable text reports. This handbook PDF is a separate documentation deliverable, not evidence that the app exports financial reports as PDF.

What the project is not

It is not a stock-trading bot, real-time price terminal, audited accounting system or autonomous investment manager. No code fetches live filings or stock prices. No local model is trained. There is no vector database, embedding model, autonomous planner, OCR engine or durable background-job service in the reviewed implementation.

A confident description is: "A full-stack, document-grounded financial research prototype using specialized processing modules, a relational database, Gemini inference and local rule-based fallbacks."

3. Web-development foundations you need

Browser, server and database

The **browser** runs the interactive interface. React displays cards, forms, buttons and charts. When the user clicks an action, browser code commonly calls a relative URL such as `/api/documents`.

The **server** runs Next.js route handlers. It verifies the session, reads request data, queries PostgreSQL and invokes agents. Gemini keys and database credentials belong here, not in browser JavaScript. A relative API URL lets the browser contact the same deployed application rather than a hard-coded localhost service.

The **database** keeps data after the browser is closed. A JavaScript variable disappears when its process ends; a committed PostgreSQL row persists. Here the database stores extracted text and derived results, not original uploaded file binaries.

HTTP and JSON

HTTP is the request-response protocol used between browser and server. GET reads data, POST creates or triggers work, PATCH updates selected fields, and DELETE removes a resource. A URL such as `/api/documents/abc` selects one document; a query string such as `?sessionId=abc` filters a collection.

JSON is a text representation of objects and arrays. For example, `{"name":"Technology comparison"}` is a session-creation request. File uploads use multipart FormData rather than JSON because the request includes a binary file. Dates are serialized into strings and PostgreSQL numeric values generally reach this code as strings.

Common responses here are 200 for success, 201 for creation, 400 for invalid input, 401 for no authenticated session, 404 for an unavailable owned record, 409 for duplicate registration email and 500 for a server failure. Not every failure path is consistently wrapped in a JSON error handler.

React and TypeScript

A component is a function describing part of a page. JSX is the HTML-like syntax inside TSX files. `useState` stores changing interface data, `useEffect` runs side effects after rendering, `useRef` retains a reference such as the chat scroll target, and `useCallback` can stabilize a callback passed to another library.

“use client” marks components needing browser-side interactivity. Most dashboard pages are client components. `src/app/page.tsx` is an async server page that checks the session and redirects to login or dashboard.

TypeScript adds compile-time types. An interface describes an expected shape, but it does not automatically validate incoming HTTP JSON or a Gemini response at runtime. This distinction is crucial: `JSON.parse(text)` as `T` is a type assertion, not a safety guarantee.

Promises, async and await

An async function returns a Promise representing future completion. `await` pauses that function until the operation settles, without blocking all other server requests. Sequential awaits run stages in order. `Promise.all` starts independent operations together and waits for all results.

The upload route starts an async function and does not await it before returning 201. This is “fire-and-forget” work inside the web process, not a durable queue. If the host freezes the process after a response, the document may remain unfinished.

Relational concepts

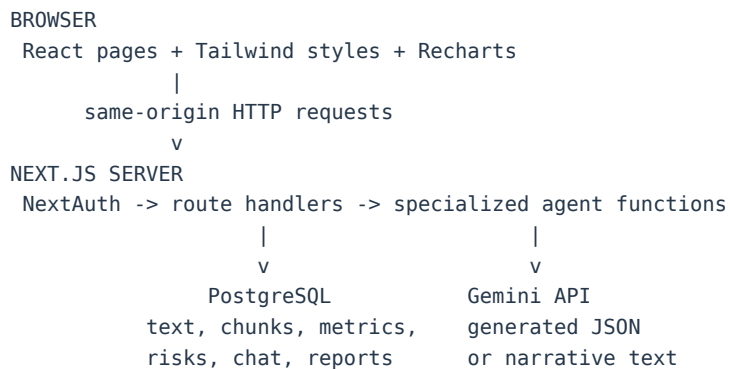
A table is a collection of same-shaped records. A row is one record. A primary key identifies it; a foreign key connects it to another table. A one-to-many relationship means, for example, one document may have many chunks.

A JOIN combines related rows. A transaction makes multiple writes succeed or fail together. An index speeds suitable lookups at a write/storage cost. Drizzle helps write typed SQL queries, but it does not automatically create authorization policies or transactions.

4. Architecture and end-to-end data flow

The actual deployment shape

This is a modular monolith: one Next.js application contains both the UI and backend routes. It connects to PostgreSQL and, when configured, Google's Gemini service. The agents are modules inside the same server runtime. They are not deployed as independent microservices and there is no inter-agent message bus.



Upload sequence

1. A signed-in user fills optional metadata and drops one file on the documents page.
2. The browser sends multipart FormData to POST /api/documents.
3. The server checks login and, if supplied, verifies ownership of the research session.
4. It reads the full file into memory and extracts text according to its format.
5. It removes NUL bytes, finds or creates a company and inserts a processing document row.
6. The route starts an unawaited pipeline and returns a 201 response containing the initial document.
7. Document Agent cleans and chunks text, saves chunks and marks the document indexed.
8. Extraction Agent attempts Gemini JSON extraction, falling back to local labeled-number parsing on error.
9. Risk Agent attempts Gemini risk extraction, falling back to keyword-based risk indicators on error.
10. The outer pipeline marks the document completed even if extraction or risk processing threw errors caught by their individual wrappers.

These stages are sequential in the background function. The initial HTTP success means the document row was created; it does not mean all analysis has finished or passed a quality test.

Research-question sequence

The UI chooses a session, POSTs the question, and immediately displays a temporary user message. The server saves the real user message, checks whether a non-demo Gemini key exists, then either returns a canned demo answer or invokes Research Agent. That agent retrieves chunks, adds metrics and risks, includes recent conversation turns and requests a Gemini answer. If that attempt fails, it constructs a local evidence summary. The server saves the assistant response and the UI replaces its temporary message with the stored pair.

Benchmark and report sequence

Benchmarking selects at least two company IDs and returns their permitted document-derived metrics and risks plus optional AI insights. The route does not insert into `benchmark_comparisons` despite that table existing.

Report creation inserts a generating report row and invokes Report Agent without awaiting completion. The agent gathers company profiles and permitted metrics/risks, generates a narrative or local substitute, updates report fields and marks it completed. The report detail page offers manual status refresh rather than a durable event subscription.

Trust boundaries

Browser input, uploaded documents and model output are all untrusted. Authentication protects a login boundary; authorization must additionally restrict each queried resource. The current code does this unevenly. Financial evidence sent to Gemini crosses an external-provider boundary, so confidential uploads need consent and data-governance review before production use.

Sources: `src/app/api/documents/route.ts`, `src/app/api/chat/route.ts`, `src/app/api/benchmark/route.ts`, `src/app/api/reports/route.ts` and `src/lib/agents/*.ts`.

5. Tools and technologies: what each one does

Core execution stack

Next.js 16.2.6 provides file-based App Router pages, API route handlers, rendering, development/build commands and production serving. Route files are named `route.ts`; UI routes use `page.tsx`; shared wrappers use `layout.tsx`. `npm dev/build` scripts explicitly use webpack even though `next.config.ts` contains a Turbopack root setting.

React 19.2.6 and **react-dom 19.2.6** provide the component model and browser rendering. This app uses local component state and `fetch`; it does not have Redux or a query-cache library.

TypeScript 5.9.3 checks interfaces, function arguments and inferred database types before execution. `strict` is enabled. The `@/*` alias maps imports to `src/*` so code can import `@/db` instead of lengthy relative paths.

Node.js runs the server and tooling. This review used Node 22.22.3 and npm 10.9.8. The repository does not pin a Node version with an `engines` field or a `version-manager` file; a supported modern Node LTS should be chosen and pinned for reproducibility.

Data, identity and AI

PostgreSQL is the relational database. It supports UUID IDs, exact numeric columns, arrays, JSONB and foreign-key delete actions. The repository does not specify a PostgreSQL server version.

pg 8.20.0 is the Node PostgreSQL driver. `src/db/index.ts` creates a Pool from `DATABASE_URL`. A pool reuses database connections; this code does not explicitly configure connection count, timeouts or provider-specific TLS settings.

drizzle-orm 0.45.2 maps typed table definitions to SQL queries. **drizzle-kit 0.31.10** is development tooling for schema changes, migrations and inspection. Runtime Drizzle uses `DATABASE_URL`, but the checked-in Drizzle Kit JSON contains a hard-coded local URL; those configurations must be aligned.

next-auth ^4.24.15 manages credential authentication and JWT-backed sessions. **bcryptjs ^3.0.3** hashes and compares passwords. `bcrypt` is not reversible encryption; the app should never need to recover the original password.

@google/generative-ai ^0.24.1 is the Gemini SDK used by the shared client wrapper. It calls a pre-trained external model; it does not train a project-specific model or provide retrieval by itself.

Reading files and displaying results

pdf-parse 1.1.1 extracts text from PDFs. The upload route deliberately requires `pdf-parse/lib/pdf-parse.js` to avoid the wrapper's test-fixture behavior under some bundling conditions. It is not OCR and does not preserve reliable table/page structure in the stored representation.

mammoth ^1.12.3 extracts raw text from DOCX. It does not render the Word document in the browser. The current catch block falls back to UTF-8 decoding the ZIP-based DOCX bytes, which is not a meaningful recovery method.

react-dropzone ^20.1.2 implements drag/drop and accepted-file UI behavior. It accepts TXT, PDF and DOCX, with one file at a time. There is no configured maxSize and browser checks are not a server security boundary.

recharts ^3.10.1 displays bar/radar charts from stored metrics. It does not calculate accounting correctness. **lucide-react** ^1.47.0 supplies icons. **react-hot-toast** ^2.6.1 displays temporary success/error notifications.

Tailwind CSS 4.1.17, **@tailwindcss/postcss 4.1.17** and **PostCSS 8.5.8** generate and transform utility-class styles. **globals.css** also supplies report styles, animation classes, scrollbars and chat-bubble colors. **clsx** ^2.1.1 conditionally builds class strings; **tailwind-merge** ^3.7.0 resolves conflicting Tailwind classes in the cn helper.

Tooling and dependencies that are not active features

ESLint 9.39.4 with **eslint-config-next 16.2.6** checks code patterns and React hooks. Lint is separate from typecheck/build. **dotenv** ^17.3.1 is available for explicit environment loading in standalone tooling; Next.js already loads supported .env files for the application.

jsPDF ^4.2.1 is used by scripts/generate-documentation-pdf.cjs to turn the older Markdown documentation into a PDF. **jspdf-autotable** ^5.0.8 is installed but not imported by the reviewed application or that script. Having these dependencies does not mean the dashboard exports reports as PDF.

framer-motion ^13.4.0 and **date-fns** ^4.4.0 are installed but have no imports in src at this snapshot. Animations use CSS and dates use Intl.DateTimeFormat. Avoid presenting unused dependencies as implemented features.

The @types packages provide editor/compiler declarations: node, pg, react, react-dom, bcryptjs, jspdf and pdf-parse. They do not independently add runtime behavior. Some are listed in dependencies rather than devDependencies, but that packaging choice does not change their purpose.

Version strings beginning with ^ are allowed ranges, not exact installed-version guarantees. package-lock.json normally fixes resolved versions; the mismatch discovered in this review currently undermines clean npm ci installation.

6. Set up the project from scratch

Prerequisites and safety

Use a terminal, Git, a supported Node LTS installation, npm, PostgreSQL with psql, and a text editor such as VS Code. A Gemini API key is optional for basic stored-data exploration but needed for live AI answers. Choose a disposable development database first; do not run seed scripts against real financial records.

Never publish DATABASE_URL, NEXTAUTH_SECRET or GEMINI_API_KEY. The example environment file contains placeholders, not working credentials. Do not prefix these variables with NEXT_PUBLIC_, which would make them eligible for browser exposure.

Step 1: obtain and enter the code

```
git clone https://github.com/Gauravcodesbyheart/multi-agent-financial-research-research-  
system.git  
cd multi-agent-financial-research-research-system  
node --version  
npm --version
```

If you already have a checkout, use that directory rather than cloning inside it. Git stores source history; npm installs libraries. They solve different problems.

Step 2: install dependencies, accounting for the known blocker

The normal reproducible command is npm ci. At this snapshot it failed under npm 10.9.8 because the lockfile's picomatch 2.3.2 did not satisfy a required 4.0.7 entry and another 2.3.2 entry was missing.

For this review, dependencies were installed without changing tracked manifests using the following temporary workaround. Ignoring lifecycle scripts reduces installation side effects; dependencies needing such scripts in a different environment may require separate review.

```
npm install --package-lock=false --ignore-scripts --no-audit --no-fund
```

Recommended permanent repair: run npm install in a controlled environment, inspect and commit the resulting lockfile, remove/recreate the dependency installation safely, and verify npm ci succeeds. Do not claim the workaround fixes reproducibility; it bypasses the broken lockfile requirement. No lockfile repair was made as part of this documentation task.

Step 3: configure local environment

```
cp .env.example .env.local  
openssl rand -base64 32
```

Use the generated random output as NEXTAUTH_SECRET. Edit .env.local with your database connection and local application URL:

```
DATABASE_URL=postgresql://YOUR_USER:YOUR_PASSWORD@localhost:5432/finresearch_dev  
NEXTAUTH_SECRET=YOUR_RANDOM_SECRET  
NEXTAUTH_URL=http://localhost:3000  
GEMINI_API_KEY=  
GEMINI_MODEL=YOUR_AVAILABLE_GEMINI_MODEL  
GEMINI_PRO_MODEL=YOUR_AVAILABLE_GEMINI_MODEL
```

Replace the model placeholders with IDs that your provider account actually supports before enabling AI. Alternatively omit those two overrides to use the source defaults, but verify that those defaults remain available. The example file names `gemini-3.6-flash` while `gemini.ts` defaults both paths to `gemini-2.0-flash`. Neither name was validated against the provider in this review. The “PRO” variable does not guarantee a different model tier.

`Next.js` loads `.env.local`, but your shell and `psql` do not automatically inherit its values. Set `DATABASE_URL` separately in a trusted shell session when running database commands, or use your database client's secure connection mechanism. URL-encode special characters in a connection-string password.

Step 4: create the database and schema

A local PostgreSQL administrator can create a dedicated development role and database through `psql` or a GUI. Grant the application only the permissions it needs. Managed PostgreSQL can also work, but its TLS and networking requirements depend on the provider.

For a brand-new empty database, the committed SQL migration is a direct way to create the schema. Confirm the target before applying it:

```
psql "$DATABASE_URL" -c 'SELECT current_database(), current_user;'
psql "$DATABASE_URL" -v ON_ERROR_STOP=1 \
  -f drizzle/0000_bizarre_enchantress.sql
psql "$DATABASE_URL" -c '\dt'
```

The migration creates 11 tables and their foreign keys. Do not repeatedly run this CREATE TABLE migration against a populated schema. Direct SQL application does not populate Drizzle's migration-history table, so choose and document a consistent migration strategy before later upgrades.

Another development workflow is Drizzle Kit push after configuring the correct database URL. The checked-in `drizzle.config.json` points to a particular local database and does not interpolate `DATABASE_URL`. A safer future TypeScript config can explicitly load environment variables and use `defineConfig`. Review schema diffs before pushing; do not treat push as a production backup strategy.

Step 5: start the application

```
npm run dev -- --hostname 0.0.0.0
```

Visit `http://localhost:3000` on your own computer. A remote preview must instead use its actual HTTPS preview URL for browser access and an appropriate `NEXTAUTH_URL`. Do not tell a remote browser to connect to the sandbox's localhost. If a development host rejects the preview origin, configure the exact allowed development origin rather than disabling security indiscriminately.

Step 6: choose demo mode or a fresh account

For a demo dataset, seed the empty development database **before registering any account**:

```
curl -X POST http://localhost:3000/api/seed
```

The current seed function skips everything if any user exists. Consequently registering first and clicking “Load Demo Data” later can produce a success response without creating demo companies. Do not delete real users just to seed; use a fresh disposable database or improve the seed logic.

The repository's public demo accounts are `demo@finresearch.ai` and `student@finresearch.ai`, both with `demo123456`. These are intentionally insecure demo credentials. Remove/disable them and protect or

remove the seed endpoint before exposing a deployment. The main demo session/documents belong to the first account, not both accounts.

If you do not need the demo, use /register, then /login, create a session and upload your own non-confidential sample. Configure a valid key to test real AI; without a key, chat returns a canned demonstration rather than live retrieval.

Step 7: validate your installation

```
curl http://localhost:3000/api/health
npm run typecheck
npm run lint
npm run build
npm start -- --hostname 0.0.0.0
```

Run start after a successful build and stop the development server first if both would use port 3000. /api/health only verifies a SELECT 1 database round trip. It does not verify schema existence, Gemini quota, parsing capability, upload jobs or authorization.

7. Beginner walkthrough and a safe demonstration

Create a small, understandable test report

Use this artificial TXT content first. Because it has short labeled lines, it isolates application behavior from PDF parsing complexity. Use decimal margins to avoid the known local percentage-normalization issue.

```
EXAMPLE ALPHA - ILLUSTRATIVE, NOT A REAL COMPANY
Annual Report 2025
BUSINESS OVERVIEW
A small educational software business.
FINANCIAL STATEMENTS
Total revenue: 100 million
Gross profit: 60 million
Gross margin: 0.60
Operating income: 20 million
Operating margin: 0.20
Net income: 15 million
Net margin: 0.15
Total assets: 200 million
Total liabilities: 80 million
Total equity: 120 million
Total debt: 40 million
Current ratio: 1.8
Debt to equity: 0.3333
Operating cash flow: 25 million
Capital expenditures: 5 million
Free cash flow: 20 million
RISK FACTORS
Supplier concentration could disrupt service delivery.
```

Create a session named “Learning financial analysis,” open Documents, supply company name “Example Alpha,” year 2025, select that session and upload the TXT file. The company association helps benchmarking/reporting find the derived values. The session association is needed for this document to appear in session research.

Open the document details. Verify its text, status, chunk count, metrics and risks. Revisit or refresh after processing; the documents page only schedules one additional refresh after three seconds, not continuous polling.

Ask precise questions

With a working model configured, ask: “What revenue and net income are reported for Example Alpha? Cite the supporting document.” Then ask: “What is the net margin, and what is the main disclosed operational risk?” Expected input arithmetic is $15 / 100 = 0.15$, or 15%. AI output must still be checked against the text.

Without a key, this route will return its hard-coded demo answer, including unrelated sample-company text. That is not a successful question-answering test. A configured key whose request fails follows a different path and may return a labeled local evidence answer.

Compare a second sample

Make Example Beta with revenue 80 million, net income 16 million and net margin 0.20, using the same year and units. Associate its document with a second company. In Benchmarking select both. Alpha is larger by revenue, but Beta has the higher net margin. A good explanation distinguishes size from profitability instead of saying one company is universally “better.”

Generate a report for both companies. Open it, manually refresh until complete, inspect the disclaimer and download the TXT. Explain that the report is built from stored metrics and risk flags rather than re-reading every original PDF page.

Presentation checklist

- Show the architecture before the UI so the audience understands the roles of browser, server, database and Gemini.
- Demonstrate one small file first, then a real searchable PDF only after verifying its contents and permissions.
- Show the raw extracted text alongside at least one displayed metric and one risk excerpt.
- Explain any fallback banner instead of pretending the model answered when it did not.
- End with known limitations and the prioritized improvement plan. Responsible scope control is a strength.

8. Document Agent and file ingestion in detail

Parsing happens before the agent

POST /api/documents reads the uploaded file using `arrayBuffer` and `Buffer.from`. PDF detection accepts the MIME type or `.pdf` extension. DOCX detection uses the filename suffix. Other files are decoded as UTF-8, so the backend does not enforce the same format allowlist as the dropzone.

For PDF, `pdf-parse` returns text and the route trims it. An empty or failed parse is replaced with explanatory placeholder text describing an unreadable/scanned PDF. That placeholder is stored and still passes through analysis. A completed status can therefore describe analysis of an error message rather than a real filing. An explicit `parse_failed` state would be more truthful.

The route strips NUL characters because PostgreSQL text cannot contain them. It saves file type as `pdf/docx/txt/unknown`, while seeded documents use `text/plain`. The original byte stream is not saved to object storage or a file path. Re-downloading the original upload is therefore not an implemented feature.

Cleaning and splitting

`processDocument` logs a running activity and normalizes CRLF newlines, reduces runs of three or more newlines to two, removes non-ASCII characters and trims the result. Removing all non-ASCII characters loses accented names, many currency symbols and non-English text. `Raw documents.content` retains the route-extracted text, while chunks contain this more aggressively cleaned version.

`chunkText` defaults to 1,500 characters with 200-character overlap. The next chunk starts 1,300 characters after the previous start. Example ranges are 0–1499, 1300–2799 and 2600–4099, subject to end-of-text truncation. These are character windows, not tokens, sentences or PDF pages.

Overlap preserves context around boundaries. A sentence crossing one cut may appear intact in the adjacent chunk. The cost is duplication and more storage. For text length $L > 0$, this loop emits $\text{ceil}(L / 1300)$ chunks at the defaults; it can emit a final chunk largely or wholly inside the preceding overlap because it keeps stepping while $\text{start} < L$. Empty text yields zero chunks.

If a caller passed `overlap >= chunkSize`, the loop would fail to advance correctly. Current callers use safe constants, but the exported helper has no argument validation. Testing reusable helpers still matters even when their current caller is safe.

Section labels and estimated pages

`extractSections` recognizes line-start headings for business overview, risk factors, management discussion, financial statements, liquidity, results of operations and notes. Content before a recognized heading is General. The heading line itself is not included in the section content.

Chunk-to-section matching estimates positions using concatenated preceding chunks. Because chunks overlap and the section representation omits heading lengths, these offsets drift. A chunk spanning several sections gets only one approximate label. $\text{pageNumber} = \text{floor}(\text{chunkIndex} / 3) + 1$ is a fabricated estimate, not a parser-reported PDF page. Never call it an exact citation page.

Chunks are inserted in batches of 50 after deleting existing chunks when the new chunk set is non-empty. The database records chunkIndex, content, section and estimated page. There is no vector column or full-text index. The document is marked indexed, with chunkCount and updatedAt changed.

Failure and reprocessing behavior

On processing failure, the function attempts to log failed, marks the document failed and rethrows. Those recovery writes can themselves fail if the database is unavailable. There is no transaction around deleting and reinserting chunks, so a partial failure can leave a partial chunk set. If reprocessing produces zero chunks, the delete block is skipped and older chunks could remain.

Sources: `src/app/api/documents/route.ts`; `src/lib/agents/documentAgent.ts`.

9. Retrieval and RAG: what is really implemented

What RAG means

Retrieval-augmented generation means retrieving supporting information and supplying it to a language model before asking it to answer. RAG is an architecture, not a synonym for embeddings. This project has a simple keyword-retrieval form of document grounding; it does not implement semantic vector similarity.

Exact search algorithm

`searchDocumentChunks` loads at most 100 chunks for one document without explicit ordering. It lowercases the query, splits on whitespace and retains words longer than three characters. Each word becomes a regular expression. For each candidate chunk, the code counts matching occurrences of every retained word and sums them. Results are sorted descending by that score and the top limit entries are returned.

For the query “revenue growth and debt,” the words revenue, growth and debt contribute. “ROE” and “EPS” would be removed because each has only three characters. A chunk saying “net sales” rather than “revenue” may score poorly even if it contains the answer. A common repeated word can outrank a better sentence.

Query words are not regex-escaped. Special characters can change matching or produce a regex error. This is a correctness and potential resource-abuse concern, distinct from SQL injection. Even zero-score chunks are returned because there is no relevance threshold.

Bounds and citation limitations

Research Agent considers the first five session documents, requests three chunks from each and sends only the first 400 characters of each selected chunk to Gemini. It exposes the first five retrieved items as citation objects, with 200-character excerpts on the AI path. Those citations are attached based on retrieval, not parsed and verified against each generated claim.

There is no global reranker across all session documents. A later document can be excluded, and early-document citations can dominate. The first 100 chunks are an unordered limited subset, not a guaranteed complete scan of a long document. No similarity confidence or claim-evidence validation is stored.

How to explain a future improvement

A stronger pipeline could preserve page/section offsets, use structure-aware chunking, create embeddings, store vectors in an appropriate index, combine vector retrieval with keyword/full-text retrieval and rerank candidates. It should keep user/session filters inside every retrieval query and require evidence for each numeric claim.

Embeddings represent text as numeric vectors so related meanings can be close even when exact words differ. They improve recall for synonyms but add model cost, storage, index maintenance and evaluation needs. They do not automatically eliminate hallucinations or authorization mistakes. These are proposed improvements, not current dependencies.

Sources: `src/lib/agents/documentAgent.ts::searchDocumentChunks` and `src/lib/agents/researchAgent.ts`.

10. Extraction Agent and financial normalization

The AI extraction path

`extractFinancialMetrics` writes a starting log and takes the first 8,000 characters of the supplied text. It asks Gemini for an object with fiscal metadata, 25 financial measurements and `raw_metrics`. The system instruction requests monetary amounts in millions of USD, percentage-like ratios as decimals and null for missing values. EPS is specifically requested in dollars per share.

The returned object is parsed as JSON, mapped from `snake_case` model keys to camelCase TypeScript fields, converted into decimal strings where present and inserted into `financial_metrics`. `companyId` comes from the argument or the document, while `fiscalYear` can fall back to document metadata. A completion log records duration.

Despite the comment “Upsert metrics,” this is a plain INSERT with no conflict target. Re-running extraction can create additional metric sets. No uniqueness constraint prevents duplicate document/year/period rows.

The local fallback

If the guarded block throws, `extractMetricsLocally` searches the original full content for labels followed by a number. `findNumber` joins alternative labels into a regex, allows up to 80 non-digit/non-minus characters before the number, supports commas and decimals, and optionally recognizes %, percent, million or billion. Billion values are multiplied by 1,000; other numbers are returned unchanged.

It extracts an annual-report-style year, defaults `fiscalPeriod` to Annual, supports a subset of fields and sets `raw_metrics.extraction_method` to local-fallback. It does not extract revenue growth, EBITDA, EBITDA margin, quick ratio or P/E on this local path. It inserts available values and logs fallback completion.

Important traps in the numeric rules

The fallback recognizes a percent suffix but does not divide by 100. “Gross margin: 25%” becomes 25, whereas `formatPercent` multiplies by 100 and displays 2500%. This is a unit-normalization bug, not merely a visual issue.

It assumes unlabeled magnitudes fit the expected unit convention. “Revenue: 1000000” could mean dollars, thousands or millions; the code cannot know. Parentheses for negative accounting values, table headers, several adjacent years and currency conversions are not handled robustly. A line using the words “gross margin” for a dollar amount may be captured as a ratio. The first matching label wins, which can pick a product's net sales rather than total-company revenue.

The AI path has different weaknesses: truncation may exclude the financial statements, JSON mode does not validate semantic correctness, and no runtime schema proves values are numbers, ratios are normalized or totals reconcile. It also requests USD normalization without a currency-rate data source.

What a quality extraction system should add

Recommended changes are field-level schema validation, explicit currency/unit/period provenance, exact source-span references, immutable extraction runs, deterministic unit conversion, amount-versus-percentage distinction and reconciliations such as gross profit / revenue versus gross margin. A null must mean unavailable, not zero. A corrected pipeline should retain both reported and normalized values.

The current broad catch can treat database/write errors as if Gemini were unavailable. A completion-log error after a successful insert may lead to a second fallback insert. Separate provider, validation and persistence failures before retrying or falling back.

Source: `src/lib/agents/extractionAgent.ts`.

11. Financial concepts and every metric family

Reading the three statements

The income statement describes performance over a period: revenue, expenses and profit. The balance sheet describes a position at a date: assets, liabilities and equity. The cash-flow statement describes cash movements over a period. Profit and cash are related but not interchangeable.

In this application, monetary statement values are intended to be stored in millions. A stored revenue of 100 means USD 100 million, and many UI components multiply by 1e6 before compact dollar formatting. A stored margin of 0.15 means 15%. Current ratio and debt-to-equity are multiples, not percentages. EPS is dollars per share, not millions.

Revenue and profitability fields

revenue: total sales recognized for a period. **revenueGrowth:** (current revenue - prior revenue) / prior revenue when periods and accounting bases match. It is undefined if the prior denominator is zero; do not silently substitute zero.

grossProfit: revenue minus cost of goods/services sold. **grossMargin:** gross profit / revenue. It measures production/service profitability before many operating costs.

operatingIncome: profit after operating expenses but before financing/tax treatment, according to the source statement's definition. **operatingMargin:** operating income / revenue.

netIncome: profit after the statement's full recognized costs, financing and tax effects. **netMargin:** net income / revenue. High revenue with a low net margin can produce less profit than a smaller, more efficient business.

ebitda: earnings before interest, taxes, depreciation and amortization. Definitions and adjustments vary. **ebitdaMargin:** EBITDA / revenue. Neither measure equals operating cash flow; working capital and capital expenditure still matter.

Balance-sheet fields

totalAssets: resources recognized on the balance sheet. **totalLiabilities:** all recognized obligations, not only interest-bearing debt. **totalEquity:** residual ownership interest; in a simple balance-sheet identity, assets = liabilities + equity.

cashAndEquivalents: cash and qualifying near-cash holdings. **totalDebt:** borrowing obligations under the document's definition. Debt is a subset of liabilities, so totalDebt and totalLiabilities are not interchangeable.

Ratios and per-share fields

currentRatio: current assets / current liabilities. **quickRatio:** a commonly used liquid-assets / current-liabilities measure, often excluding inventory and prepaid expenses; the exact definition should come from the source. The schema lacks separate current-assets/current-liabilities fields, so these ratios cannot always be independently recomputed from stored columns.

debtToEquity: debt / equity according to the selected debt definition. Near-zero or negative equity can make it difficult to interpret. **roe:** net income / average equity in a conventional analysis. **roa:** net income / average assets. Source definitions may use different denominators; very high ROE can reflect a small equity base rather than a universally superior business.

eps: earnings attributable to common shares / weighted-average shares, with basic/diluted distinctions. The schema has only one EPS column and no explicit basic/diluted basis. **peRatio:** price per share / EPS. No live price feed exists here, so the application cannot independently produce a current market P/E without document data or a new integration.

Cash-flow fields

operatingCashFlow: cash generated or used by operating activities. **capitalExpenditures:** cash spent on long-lived assets under the chosen convention. **freeCashFlow:** commonly operating cash flow minus capital expenditures, but company definitions differ. A negative sign convention for capex must be normalized before subtracting it.

Worked example and comparison discipline

For the illustrative Alpha report, revenue is 100, net income 15 and free cash flow 20, all in millions. Net margin = $15 / 100 = 0.15$, or 15%. If previous revenue was 80, growth = $(100 - 80) / 80 = 0.25$, or 25%. If Beta has revenue 80 and net income 16, its 20% net margin exceeds Alpha's, even though its sales are smaller.

Before comparing real companies, check fiscal year-end, annual versus quarterly periods, currency, units, GAAP/non-GAAP basis, industry and consolidation scope. The current benchmark does not enforce these checks. The application generally extracts values; this chapter explains financial formulas, not formulas universally calculated by the implementation.

12. Risk Agent: signals, evidence and limitations

AI path

scanForRisks logs the start and passes the first 10,000 characters to Gemini. The prompt requests 3–8 items with risk_type, severity, title, description, source_text and recommendation. Suggested types include liquidity, debt, revenue, margin, regulatory, management, market, going concern, concentration and operational risks.

The TypeScript interface permits severity critical/high/medium/low. The database column is plain varchar, so those values are not enforced as a database enum. A model item missing type/severity/title gets fallback strings; other malformed shapes are not rigorously validated.

The prompt asks for an exact source quote, but the code does not verify that the quote occurs in the source. page_reference exists in the interface and database but is not requested in the shown JSON example. Treat sourceText as candidate evidence requiring human verification, not automatically authenticated provenance.

Local path

The local checker tests five categories: business-risk/uncertainty words, debt/borrowings/interest leverage, liquidity/working-capital wording, regulatory/legal wording, and supply-chain/supplier/geopolitical wording. Each matched category produces one medium-severity indicator with a canned explanation and recommendation.

This is keyword detection, not a calibrated risk model. A company saying “we have no debt” still contains debt. A risk-free claim can trigger a flag, while a sophisticated risk phrased without these words can be missed. All local severities are medium, so the output should never be described as a rigorous relative-risk ranking.

Why the output needs a reviewer

The AI prompt's minimum of three items can encourage over-reporting when evidence is sparse. Truncation can exclude a later risk section. There is no deduplication on reruns and no independent financial anomaly calculation. A better design would permit zero supported risks, verify quotes, preserve page spans and record model/rule version and confidence or review state.

Do not confuse a flagged disclosure with a prediction of default or an investment recommendation. The app is highlighting text for review.

Source: src/lib/agents/riskAgent.ts.

13. Research Agent: context, answers and citations

Gathering evidence

answerResearchQuestion accepts sessionId, userId, question and conversationHistory. The userId argument is not used to scope its document/metric/risk queries. It loads documents by sessionId, derives distinct company IDs, retrieves up to three chunks from each of five documents, and queries all financial metrics and risks for those company IDs.

This means structured metrics can come from other sessions for the same company. They are not strictly restricted to the document IDs retrieved for the current session. The normal upload route tends to create user-associated company IDs, but that convention is not a substitute for authorization, especially given missing chat session ownership checks.

Building the prompt

The context names available documents and companies, includes up to 400 characters from each retrieved chunk, then lists revenue, net income, gross margin and ROE for metric rows and descriptions of risks. The route initially keeps ten recent messages; the agent further slices to six. Because the user message is saved before history is loaded, the current question can appear in both history and the explicit CURRENT QUESTION block.

The system instruction tells Gemini to use only provided document data and cite every claim. This is an instruction, not an enforcement mechanism. The text response is returned directly. The “reasoning” field saved separately is a short application-generated count of documents/passages/metric sets/risks, not a verified trace of the model's internal reasoning.

Citation objects

Citation objects contain documentName, section and excerpt. They are selected from retrieved chunks, not matched to individual answer sentences. A response may use information absent from the first five citation cards. A better implementation would require structured claim-to-source references and validate them against permitted source spans.

Local fallback versus no-key demo

If the Research Agent's guarded work fails, buildLocalResearchAnswer reads session documents and stored data and returns an explicitly labeled Local Research Mode answer. It includes up to ten metric rows, eight risk rows and five citation excerpts of up to 300 characters. It summarizes available data rather than solving every nuanced question.

However, when GEMINI_API_KEY is absent or starts with the demo prefix, /api/chat bypasses the agent completely and creates a hard-coded Demo Mode Response. That response mentions semantic search and sample-company figures even though neither is a verified result for the user's question. Explain this difference clearly in a demo or interview.

Sources: src/lib/agents/researchAgent.ts; src/app/api/chat/route.ts;
src/app/dashboard/research/page.tsx.

14. Benchmark Agent and Report Agent

Benchmarking logic

The API accepts companyIds and optional sessionId, checks that there are at least two entries, then loads company records and document-derived metrics/risks. Metrics and risks are joined to documents and filtered to documents owned by the current user or marked seeded. Company metadata itself is queried directly by the supplied IDs.

When AI is enabled, generateBenchmarkInsights orders metric rows by fiscalYear descending and extractedAt descending and picks one row per company. It builds a compact profile containing revenue, growth, margins, liquidity/leverage, ROE, EPS and counts of risks and critical risks. It asks Gemini for growth, profitability, health, risk and ranking commentary.

If that AI call fails, the agent ranks companies by numeric ROE, treating missing ROE as zero, and prints a labeled local comparison. That is a narrow fallback, not a comprehensive investment score. If no key is configured, the route does not call the agent: it returns the data plus a configuration message.

A subtle latest-period mismatch

The benchmark route's own metrics query is not ordered. The UI finds the first row for each company. The agent orders rows to choose its latest row. Therefore the plotted values and narrative can refer to different periods when multiple rows exist. Company listing also takes a first metric row without ordering. A centralized, explicit selected-period query would fix this inconsistency.

PostgreSQL descending sort can put null years first unless null placement is specified. Treat “latest” as an intended policy, not an absolute guarantee when fiscalYear is missing.

Report generation

POST /api/reports accepts companyIds, optional sessionId/title/reportType/additionalContext, validates that company records exist and that at least one eligible document exists across the selection, and inserts an initial report. This does not prove every selected company has an eligible document, nor does it verify ownership of a supplied sessionId.

generateAnalysisReport gathers profiles, ordered metric rows and permitted risks, then asks Gemini for executive summary, company profiles, financial performance, risk analysis, benchmarking, investment considerations, recommendations and disclaimer. The call uses DEFAULT_GEMINI_PRO_MODEL, which can be configured to the same model as other agents.

The generator is company-based: sessionId is mainly an association/logging input, not a filter limiting metrics to that session. additionalContext enters the prompt. reportType is stored but does not select different generation logic in the agent.

Extracting fields from the narrative

The executive summary is extracted with a regex looking for the exact heading # EXECUTIVE SUMMARY followed by another heading. If the heading differs, the first 500 characters are used. Recommendations are searched by a regex matching a rating word before a company name. Company names are interpolated without escaping regex metacharacters, and wording changes can prevent a match.

The report update stores full content, summary, company names, recommendations, total-company/risk/critical-risk counts, per-company risk counts and completed status. metricsComparison and the report's agentLogs JSON field are not populated by this generator. Activity events go to the separate agent_logs table.

Local reports and export

If Gemini fails, createLocalReport repeats stored metrics and risk summaries in a conservative template and advises verification. It includes the raw provider error string in its disclaimer, which may expose unnecessary operational detail. A broader failure marks the report failed.

If no session was supplied, the API passes report.id as the agent's sessionId argument for logging. agent_logs.sessionId has no foreign key, so this succeeds but is semantically misleading.

Both report-list and detail downloads create client-side text blobs and save .txt files. The detail view renders selected Markdown-like line patterns as React headings/paragraphs/lists; it is not a full Markdown renderer. An unused renderMarkdown helper exists, but it is not injected with dangerouslySetInnerHTML in the current page.

Sources: src/lib/agents/benchmarkAgent.ts, src/lib/agents/reportAgent.ts, src/app/api/benchmark/route.ts, src/app/api/reports/route.ts and report/benchmark pages.

15. Gemini integration, modes and cost model

The shared wrapper

`getGeminiClient` lazily creates and caches a `GoogleGenerativeAI` client. It rejects a missing key or a key beginning with `AlzaSyDemo`. The environment model constants are read by the module; restart the server after changing environment configuration.

`generateWithGemini` accepts a prompt, optional system instruction and model name, then returns `response.text()`. `generateJSON` additionally sets `responseMimeType` to `application/json` and parses the returned text. It does not supply a runtime response schema, token ceiling, explicit temperature, request timeout, cancellation signal or fallback model.

Retries

Errors whose text contains 503, 429, unavailable or high demand are treated as transient. `MAX_RETRIES = 3` permits the initial request plus three retries: at most four attempts. Waiting periods are 1, 2 and 4 seconds, for seven seconds of backoff plus the actual request times.

The code does not use `Retry-After`, jitter, a circuit breaker or an overall deadline. An invalid model name or invalid key may fail without retries. Error-string matching is less reliable than structured provider error handling.

Behavior matrix

Workflow	No usable configured key	Configured key but AI fails
Document chunking	Runs locally	Runs locally
Metric extraction	Attempts wrapper, then local regex fallback	Local regex fallback after failure/retries
Risk extraction	Attempts wrapper, then local keyword fallback	Local keyword fallback after failure/retries
Research chat	Canned demo answer; no Research Agent retrieval	Agent attempts a local evidence answer
Benchmark	Returns data and configure-key message	Agent can produce local ROE ranking
Report	Agent produces local report after wrapper failure	Agent produces local report after failure/retries

Cost and latency

A normal AI-enabled upload makes one extraction call and one risk call, sequentially, not six AI calls. Chat, benchmark and report each add a model call when invoked. With transient failures, each wrapper call can attempt up to four requests. Database queries, parsing and retries also contribute latency.

A conceptual cost estimate is input tokens times provider input rate plus output tokens times provider output rate, summed over successful/billable attempts under the provider's rules. This repository contains no token accounting or measured cost report. Avoid quoting a monthly price or accuracy percentage without measuring real workloads and current provider terms.

Character truncation bounds some inputs, but character count is not token count. Multilingual text, punctuation and model tokenization change the relationship. Stored metrics/risks/history can also expand prompts.

Source: `src/lib/agents/gemini.ts` and the caller modules.

16. Database design, relationships and persistence

The 11 tables

users stores identity, hashed password, role and timestamps. **research_sessions** stores a user's research folder. **companies** stores company identity and optional classification. Companies have no direct `userId`; their effective association with users is derived through documents.

documents stores metadata, extracted text, ownership, optional session/company references and processing state. **document_chunks** stores searchable text windows. **financial_metrics** stores one extracted set of measurements per inserted row. **risk_flags** stores evidence-oriented risk items.

analysis_reports stores report ownership, optional session, content and structured summary fields.

chat_messages stores ordered user/assistant turns for a session. **benchmark_comparisons** is a schema provision for saved comparisons but is not written by the current benchmark route.

agent_logs stores activity events, durations and details.

```
users
|--< research_sessions --< documents --< document_chunks
|      |      |-----< financial_metrics
|      |      |-----< risk_flags
|      |--< chat_messages
|      |--< analysis_reports
|      |--< benchmark_comparisons [schema; not saved by route]
|--< documents
|--< analysis_reports
|--< chat_messages
|--< benchmark_comparisons
```

```
companies --< documents / financial_metrics / risk_flags
agent_logs -- session/document UUID labels, no foreign keys
```

Important SQL types

UUIDs are long identifiers generated with `defaultRandom` for most primary keys. `agent_logs` instead uses an auto-incrementing serial ID. UUIDs are difficult to guess, but guessing difficulty is not authorization.

`varchar` has a length limit; text is unrestricted-length text; integer holds whole numbers. `decimal(20,2)` holds financial amounts with two decimal places, while `decimal(10,4)` holds ratios and per-share values with four decimal places. Exact decimal storage avoids some binary floating-point representation errors, but UI `parseFloat` conversions can still lose precision for very large amounts.

JSONB stores flexible objects such as `rawMetrics`, citations and `keyFindings`. text arrays store tags, report company names and recommendations. These flexible fields do not enforce a rich domain schema. Stored report company names are not company foreign keys.

Timestamps use `timestamp` rather than `timezone-aware timestamptz` in the schema. `defaultNow` fills insertion time. `updatedAt` is not an automatic trigger: callers must explicitly update it. Uploading a document does not automatically make the parent session's `updatedAt` newest.

Delete behavior

Deleting a user cascades into its referenced sessions, documents, messages, reports and benchmark rows. Deleting a session cascades into its documents and child messages/reports/benchmarks. Deleting a document cascades into chunks, metrics and risks.

Deleting a company sets `documents.companyId` to null, but cascades deletion of related metrics and risk rows. That distinction matters: text documents can remain while their structured analysis disappears. No company-delete API is currently provided.

`agent_logs` has no declared foreign keys and can remain after the source document/session is deleted. Therefore “delete document” does not guarantee complete removal of all related metadata or error excerpts. A retention/privacy policy must address logs separately.

Constraints and scale

Unique email and primary keys are enforced. There are no explicit non-primary-key performance indexes, metric-run uniqueness constraints, status/severity enums or tenant row-level security policies in the committed migration. Foreign-key references do not automatically produce all the child-column indexes needed for efficient filtering.

Drizzle relations declarations help typed relational access; they are not a replacement for SQL foreign keys and do not enforce ownership. The appendix lists each actual column and definition for detailed study.

Sources: `src/db/schema.ts`, `src/db/index.ts` and `drizzle/0000_bizarre_enchantress.sql`.

17. API reference and request contracts

How to use this reference

These are server route contracts, not a public API with bearer-token authentication. Most routes use the NextAuth browser session. The sample JSON bodies below are payloads; sending them without a valid session normally returns 401. Use browser developer tools during a local signed-in session or a properly authenticated test client. Do not copy session cookies into public documents.

Identity and infrastructure

GET/POST /api/auth/[...nextauth]: delegates NextAuth operations to authOptions. Login is normally invoked with `signIn("credentials", { email, password, redirect: false })` from the UI. Do not confuse this catch-all route with a custom token-issuing endpoint.

POST /api/auth/register: body {name,email,password}. Requires all values, lowercases email, checks duplicates, hashes with bcrypt cost 10 and returns {user:{id,email,name}} with 201. Missing values give 400, an existing email gives 409, and the catch returns 500. Server-side email format and minimum-password validation are absent; the UI's eight-character check can be bypassed.

GET /api/health: public SELECT 1 probe. Returns status, app, hard-coded version, database status, timestamp and an agents array listing five names, omitting Report Agent. Failure returns 500 with `String(error)`. It is not a comprehensive readiness check.

GET or POST /api/seed: public database mutation endpoint. Both methods call `seedDatabase` and return success even when that function skipped due to an existing user. It should not remain publicly writable in production.

Sessions

GET /api/sessions: returns {sessions:[...]} for the current user, newest updatedAt first. Each includes documentCount and messageCount from two additional queries per session. This is an N+1-style pattern.

POST /api/sessions: body {name,description?,tags?}. A truthy name is required. Returns {session} with 201 and status active. Types and string lengths are not comprehensively validated before the database.

GET /api/sessions/:id: returns {session} only if both ID and userId match; otherwise 404.

PATCH /api/sessions/:id: accepts name, description, status and tags through an allowlisted object; always updates updatedAt. Returns {session} or 404. Status is not constrained to a validated enum.

DELETE /api/sessions/:id: scoped by ID and current user. Returns {success:true} even if no row matched. Cascades can delete attached documents and analysis.

```
{ "name": "Technology comparison", "description": "Same-year  
research", "tags": [ "technology", "annual" ] }
```

Documents and companies

GET /api/documents?sessionId=...: returns {documents:[{document,company}]} for the current user, optionally filtered to a session. It selects the whole documents table object, so the response can include large content text even when the list UI only needs metadata.

POST /api/documents: multipart fields file, sessionId?, documentType?, fiscalYear?, companyName?, ticker?. File is required. A supplied session must be owned. companyName is trimmed; matching uses exact name among companies linked to this user's documents. Ticker is not the lookup key. Returns {document} with 201 before analysis finishes. Unhandled validation/database issues are caught and returned as a 500 error containing the message.

GET /api/documents/:id: verifies ownership and returns {document,metrics,risks,chunks}. Chunk response includes chunkIndex and section, not full chunk text. Missing/not-owned gives 404.

DELETE /api/documents/:id: owner-scoped deletion; returns {success:true}; removes dependent chunks/metrics/risks through database cascades.

GET /api/companies: lists seeded companies plus those linked to the current user's uploads. Each includes a first metric row, riskCount and documentCount. The per-company subqueries are company-wide, not joined back to owned documents, and have no metric ordering. Three extra queries per company make this another scaling hotspot.

Metrics, risks and dashboard

GET /api/metrics: optional documentId or comma-separated companyIds. documentId takes precedence. Returns {metrics:[{metrics,company}]}. Requires login but does not scope the query to the user's documents; without filters it returns all rows. This is a significant tenant-isolation gap.

GET /api/risks: analogous documentId/companyIds behavior; returns {risks:[{risk,company}]}]. Reads a severity parameter but does not use it. The UI applies severity/type filtering locally. This route also lacks ownership filtering.

GET /api/dashboard: returns stats, recentSessions, recentReports, recentAgentActivity and metricsChartData. Document/session/report counts and recent owned lists are user-scoped, but company/risk counts, the ten recent logs and up to twenty joined metric rows are global. An authenticated user may see other users' derived information or logged question details.

Chat

GET /api/chat?sessionId=...: sessionId is required, otherwise 400. Returns {messages} ordered oldest first. It checks login but does not verify ownership of the supplied session.

POST /api/chat: body {sessionId,content}; both required. Saves a user turn, generates demo/AI/fallback content, saves an assistant turn and returns {userMessage,aiMessage}. There is no check that the session belongs to the current user. Many model failures are represented as assistant messages in a 200 response rather than an HTTP error.

```
{"sessionId":"YOUR_OWN_SESSION_UUID","content":"What revenue is reported, and where?"}
```

Benchmark and reports

POST /api/benchmark: body {companyIds:[id1,id2],sessionId?}. Fewer than two entries gives 400. Returns {metrics,companies,risks,insights}. It filters metric/risk documents to owner-or-seeded; metadata lookup and optional log session are not equivalently validated. Array shape, duplicate IDs and maximum selection size are not robustly checked. It does not persist the comparison.

GET /api/reports: returns {reports} owned by the current user, newest first. Whole report rows include potentially large text.

POST /api/reports: body {companyId?,title?,sessionId?,reportType?,additionalContext?}. Empty company selection gives 400. Selection must refer to existing companies and have at least one eligible document collectively. Returns {report} with 201 in generating state. Optional session ownership is not checked, and report creation does not wait for completion.

GET /api/reports/:id: owner-scoped {report}, otherwise 404. **DELETE /api/reports/:id:** owner-scoped removal and {success:true}, including when no row matched.

```
{"companyId":["COMPANY_A_UUID","COMPANY_B_UUID"],"title":"Illustrative annual comparison","reportType":"comprehensive","additionalContext":"Emphasize units, missing data and source limitations."}
```

There is no API for job cancellation, reprocessing, saved benchmark retrieval, original-file download, report PDF export, password reset or account deletion in this snapshot.

18. Frontend pages, state and formatting

Navigation and shared layout

The root server page checks `getServerSession` and redirects to `/dashboard` or `/login`. `RootLayout` provides metadata, global CSS, `SessionProvider` and `Toaster`. `DashboardLayout` uses `useSession`, hides content while loading and redirects unauthenticated visitors from the client. It supplies a collapsible mobile sidebar and sign-out action.

This client-side layout gate improves user experience but is not the security boundary. A user can call API routes without opening the page. Every API must authorize independently. No middleware-based global authorization layer is present.

Each screen's job

Login: credential sign-in, toast errors and a button filling the public demo account. **Register:** name/email/password form and a client-side minimum password check. Neither screen implements password reset, email verification or MFA.

Dashboard: counts, quick actions, recent sessions/reports/activity and stored-metric summaries; it can call the seed endpoint. Counts may mix owned and global data, as noted in the API reference.

Research Sessions: fetches owned sessions, supports search, create, edit and delete. Comma-separated tag input is converted to an array. **Session Detail:** fetches session metadata and its document list in parallel; links lead to `documents/research/reports`.

Documents: metadata form, one-file dropzone, local filename/company search, status icons and delete. **Document Detail:** overview, metrics, risks and extracted-content tabs. It displays parsed text, not an embedded original PDF viewer.

Companies: cards show company metadata, first available metrics and risk/document counts. Links include `companyId` query parameters, but the destination `metrics/risks/benchmark` pages do not consume those parameters in the reviewed code.

Financial Metrics: revenue/EBITDA, margins, ratios and cash-flow bar charts plus a radar chart and table. **Risk Analysis:** severity summary, severity/type dropdowns and locally sorted risk cards.

Benchmarking: company checkboxes, POST action, comparison bars, table and narrative.

Research Agent: session dropdown, suggestion/question input, optimistic user messages, loading state, citation expansion and auto-scroll. It chooses the first loaded session by default rather than reading the `sessionId` query string. **Reports:** generation form, company/session selection, cards and TXT download. **Report Detail:** status refresh, summary, recommendations and simple formatted report lines.

Docs: static tabbed explanation. Its download action points to `/DOCUMENTATION.md` and saves a Markdown file, despite a "Download PDF" tooltip. Older static documentation is not guaranteed to match current code.

Refresh and query-string behavior

No WebSocket or server-sent-events infrastructure is implemented. Uploads schedule one extra fetch after 3 seconds. Report detail exposes manual refresh; several pages fetch only when mounted. A browser refresh can be necessary to observe completed work.

Session detail links attach `?sessionId=...`, but the target documents/research/reports pages do not automatically initialize their selection from it. Explain to beginners that they must select the session in the form/dropdown; the link alone does not reliably establish context.

Utility functions

`cn` combines `clsx` and `tailwind-merge`. `formatCurrency` formats dollar values and optionally K/M/B/T abbreviations; it does not itself convert stored millions into dollars. `formatPercent` multiplies a decimal by 100. `formatNumber` uses locale separators, `formatDate` uses `Intl.DateTimeFormat`, `getInitials` builds up to two initials, and `truncate` shortens long text. `getSeverityColor/getStatusColor` map known strings to style classes, with neutral fallbacks.

Many callers pre-convert null to "0", bypassing helpers' N/A behavior. That can display missing revenue as \$0 and missing ratios as zero. The metrics chart tooltip appends M to non-margin values, including ratio mode. The radar caps/scales values and takes absolute ROE, so it is a stylized visualization rather than a statistically comparable risk score.

Source references are the corresponding `page.tsx` files and `src/lib/utls.ts`.

19. Authentication, authorization and security review

What is already present

Registration hashes passwords with bcrypt cost 10 and lowercases email. Login queries the lowercased email and compares the hash. NextAuth uses JWT sessions with a 30-day maxAge. Its JWT callback adds user ID and role, and its session callback makes them accessible to UI/server session consumers. Type augmentation in `src/types/next-auth.d.ts` documents these fields for TypeScript.

Many document, session and report detail routes correctly combine resource ID and user ID in their SQL filters. Metric/risk access in benchmark and report generation includes owner-or-seeded document filtering. These are useful foundations but not a complete tenant-security design.

Priority 0: repair before public multi-user deployment

Cross-user data access: `/api/chat` GET/POST trusts `sessionId` without verifying ownership. `/api/metrics` and `/api/risks` can return all users' rows. `/api/dashboard` exposes global logs and metrics. Research Agent queries structured data by company without using its `userId` parameter. Fix by centralizing owned-session/owned-document checks and applying them to every query, including nested agent queries and citations.

Public seeding and fixed demo credentials: `/api/seed` mutates the database with both GET and POST and no authentication. Disable it in production or make it an explicit authenticated administrative operation with environment guards. GET should not mutate state.

Predictable session-secret fallback: `authOptions` falls back to the literal `financial-research-secret` if `NEXTAUTH_SECRET` is missing. Fail fast instead. Generate a strong deployment-specific secret and require HTTPS with appropriate cookie/proxy configuration.

Priority 1: input, privacy and resource controls

Validate payloads on the server: UUIDs, array contents/lengths, accepted statuses/severities, email shape, password length, fiscal year and string sizes. Add login/register/API rate limits and cost limits on AI-triggering actions. A user's role is stored but is not enforced as a broad authorization policy; role data alone is not RBAC.

Enforce upload size/type on the server, inspect content signatures, reject unreadable files clearly and protect parsers from resource exhaustion. Reading the full upload and sending excerpts to an external provider requires retention and confidentiality policies. Add deletion handling for logs and backups as appropriate.

Do not return raw provider/database errors to users. Upload, health, seed, chat/benchmark paths and local report content can reveal operational details. Keep detailed diagnostics server-side with a correlation ID and sanitized client messages.

Prompt injection and model output

Uploaded reports and additionalContext can contain instructions trying to redirect the model. The current prompts do not create a robust separation between instructions and untrusted evidence. The model has no arbitrary code-execution tools here, which limits one class of impact, but answers can still be manipulated or include data improperly included in context.

Use clear evidence delimiters, narrow structured outputs, runtime validation and claim-source checks. Authorization must happen before information reaches the prompt; asking the model not to reveal data is not a substitute.

React renders report strings as text/JSX in the active implementation, which avoids directly injecting arbitrary HTML. Do not falsely report a current dangerouslySetInnerHTML vulnerability based on the unused Markdown helper. If a full Markdown renderer is added, keep raw HTML disabled or sanitized.

Honest assessment

These are source-based findings, not a completed penetration test or a claim that exploitation was demonstrated. The app is appropriate as a supervised prototype/demo with non-sensitive data, but should not be called production-ready for private multi-tenant financial research until these issues are fixed and verified.

20. Reliability, performance and consistency

Jobs and state transitions

Document states are pending by schema default, processing on upload, indexed after chunking, then completed or failed. Reports start generating and become completed or failed. Log events use running/completed/failed independently. Completion is a workflow status, not a quality score; fallbacks can still be completed, and swallowed extraction/risk errors do not block the final document update.

An unawaited Promise is not a persisted job. A process restart, deployment or serverless response lifecycle can interrupt work with no automatic recovery. There is no queue, worker lease, retry schedule, cancellation, heartbeat or stalled-job reaper.

A recommended design stores a job row transactionally with document creation, has a separate worker claim it, records per-stage states, uses idempotency keys and retries only safe operations. A long-lived host reduces some process-lifecycle risk but still does not provide durability by itself.

Transactions and idempotency

Upload creation, company lookup/creation and downstream writes are separate operations. Concurrent uploads can create duplicate company rows because name matching has no unique owner/name constraint. Reprocessing can duplicate metrics and risks. Report creation can leave generating rows if the background work never runs.

Idempotency means repeating a request does not create unintended duplicate effects. Consider document content hashes, unique extraction-run keys, transactional replace/upsert logic, and a versioned analysis-run table. A transaction alone is not a complete job system, but it prevents certain partial-write states.

Query efficiency

Session lists make two count queries per session; company lists make three queries per company. Research does per-document retrieval, loading up to 100 chunks into application memory to score them. Several list endpoints select full document/report text, increasing response size. Most collections have no pagination.

Recommended optimizations include grouped count queries, metadata-only selects, cursor pagination, indexes on owner/session/document/company/createdAt combinations, and database-backed full-text or vector retrieval. Measure query plans and workloads before choosing indexes blindly.

Connection and runtime resources

The pg Pool has no explicit app-specific cap or timeout. Scaling web instances multiplies pools and can exhaust database connections. Large file buffers consume memory. Retry delays increase latency. Provider requests have no overall timeout, so requests can outlast hosting limits.

Production observability should separate parse time, retrieval time, AI latency, token/cost estimates, fallback rates and database latency. Current agent_logs stores action, status, details and duration but is neither a durable queue nor a complete tracing system.

21. What was tested and how to test properly

Checks executed during this review

Environment: Node 22.22.3, npm 10.9.8, Linux sandbox. No project database/Gemini credentials were configured.

Check	Actual result	Interpretation
npm ci --ignore-scripts --no-audit --no-fund	Failed: picomatch lockfile mismatch	Clean reproducible install needs repair
npm install --package-lock=false --ignore-scripts --no-audit --no-fund	Succeeded; 517 packages installed	Temporary review install; lockfile unchanged
npm run typecheck	Passed	Static TypeScript checks succeeded
npm run lint	0 errors, 6 warnings	Not warning-free; see breakdown below
npm run build	Passed, Next.js webpack production build	Compilation and page generation succeeded
Database, upload and Gemini integration	Not run	No claim of end-to-end correctness

The six lint warnings were three unused eslint-disable directives and three React effect dependency warnings. The directives occur twice in the document upload route and once in the metrics page. Hook warnings occur in report detail and twice in research. Generated next-env.d.ts and tsconfig.tsbuildinfo changes from verification were restored; application code was not altered for this documentation task.

No project test suite or npm test script was found. Passing build/typecheck does not detect data leakage, wrong percentages, provider behavior or partial background processing.

Proposed unit-test suite

Test chunkText with empty/short/exact-boundary/long text and invalid overlap. Test extractSections with heading variants, Unicode and headings inside a chunk boundary. Test retrieval with synonyms, short acronyms, regex punctuation, zero matches and over 100 chunks.

For local metrics, cover million/billion, percent/decimal, commas, negative parentheses, multiple years and missing values. A regression test should demonstrate that 25% normalizes to 0.25 after a fix. Verify money, EPS and ratios use different unit rules. Test financial formatters with null, zero, negative, invalid and very large values.

Test AI wrapper behavior with a mocked client: success, malformed JSON, 429/503 retries, non-transient failure and exhausted retries. Verify four maximum attempts and the intended delay policy without waiting in real time by using fake timers.

Proposed integration/security tests

Use an isolated PostgreSQL database and two users, A and B. Create a session, document, metrics, risks and chat for each. For every route, try reading/writing/deleting A's IDs while authenticated as B and assert no data is returned or modified. Include aggregate dashboard counts/logs and model context construction, not just detail routes.

Test upload rollback/partial failure, company-creation races, rerun duplication, empty PDF extraction, DOCX parser failure and source deletion while an analysis job is active. Test seeded/public visibility separately from private documents.

Proposed end-to-end acceptance tests

Use a browser tool such as Playwright (not currently installed in this repo) to register/login, create a session, upload a labeled fixture, inspect derived output, ask a question, generate/download a report and delete the workspace. Run missing-key and mocked-provider-failure scenarios separately.

For AI evaluation, build a small licensed or synthetic gold dataset with known financial values and source spans. Measure numeric accuracy, unit/period accuracy, missing-value precision, risk precision/recall and citation faithfulness. Record model/version, prompts and extraction run IDs. Do not invent a 95% accuracy or “real-time” performance claim from a successful UI demo.

22. Troubleshooting guide

Installation or build problems

npm ci rejects picomatch: this is the observed manifest/lock resolution mismatch, not a Gemini problem. Use the documented temporary installation only for review, then regenerate/review the lockfile and re-test clean installation for a permanent fix.

Module missing: verify you are in the repository root and dependencies installed successfully. **Type errors after library changes:** check locked versions and run typecheck. **ESLint warnings:** distinguish warnings from errors and fix dependency arrays intentionally, not by suppressing all rules.

Database and authentication problems

Connection refused or timeout: PostgreSQL may not be running, the host/port can be wrong, provider networking can block the app, or required TLS can be missing. Check the database connection independently. **Relation does not exist:** connection works but the schema was not applied to that database.

Login fails after registration: verify normalized email, password, matching database environment and secret/URL configuration. `authorize` catches database errors and returns null, so a database outage can look like invalid credentials. Consult server logs rather than repeatedly changing passwords.

Demo login fails: seed before registration in a fresh dev database. Existing users cause seeding to skip. The `/api/seed` success message does not prove records were inserted.

Document and extraction problems

Upload says success but is processing forever: background work may be stopped, logs may indicate a database/provider failure, or the UI may simply not have refreshed. Inspect document state and activity, then improve durable execution rather than adding arbitrary browser delays.

Completed document has no metrics: the text may be empty/placeholder, financial statements may occur after the 8,000-character excerpt, labels may not match, or extraction errors may have been swallowed. Inspect `documents.content` before blaming the model.

Garbled DOCX: the local catch may have decoded ZIP bytes as UTF-8. Re-export to TXT or fix parsing; do not analyze the garbage as financial evidence. **Scanned PDF:** run OCR in a separate trusted workflow and upload a searchable result; no in-app OCR exists.

Margin is 2500% instead of 25%: inspect `raw_metrics` for local-fallback and normalize percentage units. Do not just divide every number by 100; current ratios and dollar amounts need different rules.

Research, charts and reports

Question returns Apple/Microsoft content for another company: likely the no-key canned demo route. Configure a real available model or implement a grounded missing-key local path.

Acronym question retrieves poor evidence: ROE/EPS are dropped by the >3-character filter.

Relevant evidence missing: retrieval caps and excerpt truncation may exclude it. **Chart differs from narrative:** unordered route/UI row selection can disagree with the agent's sorted metric selection.

No company available for comparison: supply companyName during upload. A document with companyId null can still be read, but company-based downstream queries may omit its metric/risk rows. **No session document in chat:** select a session at upload; attaching a query string to navigation does not initialize the target form.

Report remains generating: check server logs and manually refresh. **Report download is not PDF:** current export is intentionally TXT in code, regardless of installed PDF dependencies. **Risk severity query has no effect:** the backend reads but ignores severity; the page filters locally.

23. Deployment and operations without overclaiming

Minimum deployment components

You need a Node-capable host running the built Next.js application, a reachable PostgreSQL database, private environment variables and HTTPS. The repository contains no Dockerfile, container orchestration, CI workflow or production job worker. Static-file-only hosting cannot run these API routes or database operations.

Build with `npm run build` and serve with `npm start`. Configure `NEXTAUTH_URL` to the public origin and `NEXTAUTH_SECRET` to a strong private secret. Supply a database URL with the provider's required TLS settings and use currently supported Gemini model IDs. Browser fetches already use relative `/api` URLs, which is suitable for same-origin hosting.

Do not assume a serverless deployment is durable

The upload and report workflows keep working after sending the HTTP response without a supported durable queue. Some hosting environments terminate/freezes that work. Before deploying to any platform, understand its request timeouts and background-work lifecycle. Prefer a queue/worker architecture for reliable long-running analysis, regardless of provider branding.

No claim of free forever, unlimited quota or guaranteed support is made here. Hosting plans, model availability and pricing change. The older documentation's deployment suggestions should be checked against both current provider terms and this project's actual execution model.

Production-readiness checklist

- Repair and verify the lockfile; pin Node and document PostgreSQL compatibility.
- Fix all cross-user data scopes and enforce optional-session ownership.
- Disable public seeding and remove demo credentials; require a non-default auth secret.
- Validate inputs and uploads; add rate, size, timeout and AI-cost controls.
- Introduce durable jobs, transactional writes and idempotent analysis runs.
- Validate financial units, source evidence and model outputs before presenting results.
- Add multi-user integration tests, AI evaluation fixtures and CI checks.
- Add controlled migrations, backup/restore drills, monitoring and sanitized errors.
- Document provider data sharing, retention, deletion and permitted upload content.

Back up PostgreSQL using an appropriate provider/`pg_dump` workflow and test restoration into a separate database. A backup not tested for restore is an unverified assumption. Keep secrets separate from source backups and remove sensitive text from broad application logs.

24. Implementation reality versus common project claims

Claims you can defend

The project is full-stack and document-oriented. It has six specialized agent modules, credential authentication, PostgreSQL persistence, document chunking, model-assisted extraction/risk analysis, keyword-grounded chat, comparisons, report generation and deterministic fallback paths. The code compiles and typechecks in the reviewed environment after the installation workaround.

Claims you should qualify or avoid

“Five agents”: older README/docs and health output list five, but reportAgent.ts makes six specialized modules. Say six modules, with document/extraction/risk in the upload pipeline and the other three triggered separately.

“Semantic/vector search”: not implemented; retrieval is regex-count keyword scoring over stored text. **“Exact citations”**: source cards are approximate retrieved excerpts; claim-level matching and exact PDF pages are not verified.

“All documents fully analyzed”: extraction uses 8,000 characters, risk uses 10,000, research uses at most five documents and selected short excerpts. **“Every completed result is AI-generated”**: local fallbacks can mark completion, and completed upload status can mask stage errors.

“PDF report export”: financial reports download as TXT; the separate documentation generator creates PDF. **“Real-time streaming”**: API calls return complete responses, with limited/manual refreshing; no token streaming or event channel exists.

“Autonomous multi-agent orchestration”: application code invokes functions; there is no planning/replanning loop, tool registry or agent-to-agent negotiation. **“Trained our own model”**: Gemini is external inference with prompts, not project training.

“Secure multi-user platform”: login exists but authorization gaps remain. **“Saved benchmark history”**: the table exists but the route never inserts a comparison. **“Live financial data”**: seed/upload data only; no live data connector.

“Production-ready with tested accuracy”: no automated project test suite, live provider validation or measured finance benchmark was found. **“Demo data is authoritative”**: seed documents and metrics are hard-coded educational fixtures. For example, the seeded Apple text repeats the same net-sales number for 2023 and 2022 while describing a decrease; that internal inconsistency alone warrants verification against original filings before reuse.

Strengths to emphasize instead

Separation into modules makes responsibilities understandable. PostgreSQL offers relationships and structured reporting. Prompts are centralized enough to inspect. Source excerpts and explicit local-mode labels create a starting point for transparency. The UI supports a complete learning workflow. Honest evaluation and prioritized engineering improvements are more credible than inflated feature claims.

25. Roadmap: improve the project in a sensible order

Phase 1: correctness and isolation

Centralize authorization helpers and add two-user tests before adding features. Repair the lockfile, require secrets, restrict seeding and validate requests. Fix percentage normalization and null-versus-zero display. Reject parse failures instead of analyzing placeholders. Make selected fiscal period/currency explicit across metrics, benchmarks and reports.

Success criterion: one user cannot access another user's documents, derived data, chat or logs; the same source value displays consistently everywhere; a clean install succeeds.

Phase 2: trustworthy evidence and persistence

Preserve parser page references and character spans. Add analysis-run records with model/prompt versions, status, source hash and field-level provenance. Validate JSON with a runtime schema library. Verify quotes and numeric relationships. Use transactions and uniqueness constraints for safe reruns.

Success criterion: every displayed metric can be traced to a source location, unavailable data remains unavailable, and retrying does not multiply results.

Phase 3: durable execution and observability

Introduce a real job queue and worker, stage-level progress, bounded retries, timeouts, cancellation, stale-job recovery and structured logs. Add query pagination/indexes and token/cost telemetry. Avoid using raw provider exceptions as user-visible report text.

Success criterion: a process restart does not permanently abandon a document, and operators can explain where a failed job stopped.

Phase 4: retrieval and product refinement

Improve retrieval with full-text/hybrid search, embeddings if justified, sentence/section-aware chunking and reranking. Add verified claim citations, better history budgeting, continuous status refresh and URL-driven UI selection. Persist benchmark snapshots and align narrative/chart periods. Add original-file storage only with access controls and retention design.

Success criterion: retrieval evaluation demonstrates better evidence recall and source faithfulness, not merely a more fashionable technology stack.

Phase 5: advanced features

Possible later additions are OCR with page fidelity, table-aware extraction, approved live-filing connectors, valuation modules using current market data, complete report PDF export, role-based collaboration and accountant review workflows. Each adds security, cost and evaluation obligations. Add them only after the foundation is dependable.

26. Presentation scripts and design-defense answers

A 30-second explanation

“FinResearch AI is a full-stack financial research workspace. Users upload reports, and specialized modules divide the text, extract financial values and flag risks. Users can compare companies, ask document-grounded questions and generate reports. The application uses Next.js and React, PostgreSQL with Drizzle, NextAuth for login and Gemini for AI inference, with local fallback logic. It is an educational prototype, and I can explain its retrieval limits and the work needed for secure production deployment.”

A two-minute explanation

“Financial research usually requires reading documents, copying figures into spreadsheets, identifying risks and writing a narrative. My project joins those activities into a session-based workflow. The browser sends a file to a Next.js route. The route parses it, stores its text and starts a three-stage document pipeline: chunking, financial extraction and risk scanning. PostgreSQL persists the intermediate results so the dashboard, benchmark and reporting modules can reuse them.

The Research Agent retrieves keyword-relevant chunks, combines them with stored metrics and risk flags, and supplies that context to Gemini. A separate Report Agent produces a longer summary. These are specialized functions, not independent autonomous services. The current retrieval does not use embeddings, and citations still require verification.

The main engineering trade-off is simplicity versus reliability. Keeping the modules in one app makes development accessible, but fire-and-forget jobs are not durable, and several data-access routes need stronger user scoping. My next steps are authorization tests, unit normalization, runtime response validation, durable workers and better evidence retrieval. I would not claim investment-grade accuracy without a labeled evaluation dataset.”

Why these design choices are reasonable

Why Next.js rather than separate React and Express apps? One routing/deployment structure reduces prototype complexity and lets frontend/backend share TypeScript shapes. A separate worker is still desirable for long jobs. Framework choice does not solve data authorization automatically.

Why PostgreSQL rather than only a vector database? Users, sessions, ownership, financial rows and reports have relational constraints and reporting needs. A vector index could complement PostgreSQL for retrieval, but would not replace its transactional role.

Why multiple modules instead of one giant prompt? Extraction, risk analysis, retrieval and writing have different inputs/outputs and failure modes. Separation supports reuse and targeted testing. In this version “multi-agent” means role-specialized modules with application-controlled execution.

Why local fallbacks? They retain limited utility when AI is unavailable and make demo workflows possible. They trade expressive understanding for simple deterministic rules, so provenance and mode labels are essential. The fallback must never silently imply the same accuracy as AI or human review.

Why not immediately add a more powerful model? Model quality cannot repair wrong user scopes, inconsistent units, stale metric selection or missing source text. Fix data and evaluation before spending more on inference.

27. Interview and viva question bank

Fundamentals

- 1. What is the project's input-to-output pipeline?** File -> parsed text -> document row -> cleaned overlapping chunks -> extracted metrics and risk flags -> charts, chat, benchmarks and reports. Parsing is in the route; chunking is in Document Agent.
- 2. How many agents are there?** Six source modules: Document, Extraction, Risk, Research, Benchmark and Report. Older documentation lists only five. Gemini is a shared provider wrapper, not a seventh specialized agent.
- 3. Are the agents autonomous?** No. TypeScript route/function calls control their order. There is no planner, message bus or self-directed tool-use loop.
- 4. What runs on the client versus server?** Interactive React pages, chart rendering and file selection run in the browser. Session checks, database queries, document parsing and Gemini calls run on the Next.js server.
- 5. Where is data persisted?** PostgreSQL stores extracted text, chunks, structured results and chat/report data. Original file binaries are not persisted by the upload path.
- 6. Why use TypeScript?** It catches many shape/argument errors during development, but cannot prove runtime JSON or AI output is valid without explicit validation.

Documents and retrieval

- 7. How are PDFs handled?** pdf-parse extracts selectable text; failures become stored placeholder text. Scanned PDFs require OCR outside the app.
- 8. How are DOCX and TXT handled?** Mammoth extracts DOCX raw text; TXT is decoded as UTF-8. The DOCX failure fallback decodes raw bytes, which is unreliable.
- 9. Why split text into chunks?** Smaller units support retrieval and reduce prompt size. Overlap preserves boundary context at the cost of duplication.
- 10. What are the chunk settings?** 1,500 characters per chunk and 200 characters overlap, yielding a 1,300-character step. They are not model tokens.
- 11. Is pageNumber exact?** No. It is $\text{floor}(\text{chunkIndex} / 3) + 1$, an estimate unrelated to true PDF pagination.
- 12. What is your retrieval algorithm?** Load up to 100 document chunks, count regex occurrences of query words longer than three characters, sort by count and return the top few.
- 13. Do you use embeddings or a vector database?** No. Those are possible future improvements, not installed retrieval infrastructure.
- 14. Can the search miss ROE or EPS?** Yes, those three-letter tokens are filtered out. Synonyms and evidence after the candidate limit can also be missed.
- 15. What is RAG in this project?** Retrieved text is supplied as context to Gemini before answering. It is keyword-grounded RAG-like behavior rather than semantic vector retrieval.

16. Are citations verified per claim? No. Citation cards come from selected retrieved chunks. A stronger implementation would link and verify every claim against a source span.

AI and financial logic

17. Did you train Gemini? No. The app uses a pre-trained provider model through API inference and task-specific prompts.

18. What model is used? Model IDs come from GEMINI_MODEL and GEMINI_PRO_MODEL, otherwise both source defaults are gemini-2.0-flash. The example env names a different model; availability must be checked for the account.

19. How are JSON outputs made reliable? The wrapper requests application/json and parses it, but this is only syntactic parsing. Runtime schema and financial validations are not yet implemented.

20. What happens on 429 or 503? The wrapper recognizes these as transient using error text and retries up to three times after the first attempt, waiting 1, 2 and 4 seconds.

21. Does every failure invoke a fallback? No. Different routes behave differently, and database failures can also break fallback writes. No-key chat bypasses the agent and uses canned demo content.

22. Why can extraction miss values at the end of a report? Gemini extraction receives only the first 8,000 characters. The risk prompt receives the first 10,000.

23. What are the units? Statement amounts are intended as USD millions; percentage ratios are decimals; EPS is dollars per share; liquidity/leverage ratios are multiples.

24. What is the known percentage bug? Local parsing returns 25 for "25%" without division by 100, while display formatting multiplies by 100. That can produce 2500%.

25. Is null the same as zero? No. Null means unavailable, zero is a known numeric value. Some chart callers currently conflate them and should be corrected.

26. Do you compute every ratio from raw statements? No. The agent primarily extracts reported or model-derived values, and the schema lacks some denominators needed for independent calculation.

27. Why might a high ROE be misleading? A small or negative equity base and leverage can distort interpretation. The local benchmark's ROE ordering is only a limited heuristic.

28. Does a risk flag mean the company will fail? No. It is a candidate disclosure/signal. Local rules can flag negated or generic wording, and AI quotes require verification.

29. Do you fetch current stock prices? No. There is no live market-data or filing connector.

30. How do you prevent hallucinations? Current prompts ask for source-only answers, but that is not prevention. Better retrieval, output validation and claim-level evidence checks are recommended.

Backend and database

31. What are the main tables? Users, research sessions, companies, documents, chunks, financial metrics, risk flags, reports, chat messages, benchmark comparisons and agent logs.

32. Why store numbers as PostgreSQL decimals? Exact decimal storage suits financial values better than binary floating point, though UI conversions still require care.

33. What is a foreign-key cascade? Deleting a parent can automatically delete child records. Session deletion can remove its documents and associated derived results.

34. Are agent logs deleted with documents? Not automatically; their UUID fields have no foreign-key constraints. This matters for retention and privacy.

35. Is extraction an upsert? No. Despite a comment, it inserts a new metric row. Reruns can create duplicates.

36. Are benchmarks persisted? The schema has `benchmark_comparisons`, but the current route returns a response without inserting it.

37. How is a company's latest result selected? Agents sort year/time and choose a row, but UI/API paths are inconsistent and nullable years complicate ordering. This needs a shared explicit period-selection rule.

38. Why can a session report include data from another session? Report generation queries by selected company and permitted document ownership, not strictly by session document IDs.

39. What is the N+1 query problem here? Listing sessions/companies triggers extra per-item count/metric queries, so query count grows with the collection size.

40. Why not store all data in JSON? Relational columns support constraints, joins and aggregations; JSONB is useful for flexible adjunct metadata but should not replace every explicit contract.

Security and reliability

41. How are passwords protected? bcrypt hashing at cost 10, checked with `bcrypt.compare`. There is no password decryption.

42. Authentication versus authorization? Authentication establishes the user; authorization decides whether that user may access a specific session/document. Some routes implement the first without enough of the second.

43. What is the most serious security gap? Cross-user scopes in chat, metrics, risks and dashboard/log data, plus missing nested query ownership checks. Fix these before public multi-user use.

44. Does role mean RBAC is implemented? No. Roles are carried in the session but broad role-based permissions are not enforced.

45. Why is a default auth secret bad? A predictable secret undermines trust in sessions. The app should refuse startup if a strong private secret is absent.

46. Why is public seeding unsafe? It changes data without authentication, even through GET, and creates known-password demo users. Production should disable or strictly guard it.

47. Is a fire-and-forget Promise a queue? No. It lacks persisted state, worker recovery and guaranteed execution after response/process termination.

48. Does completed mean fully accurate? No. It means a path reached that state. Fallbacks and swallowed stage errors can still end in completed.

49. How would you make retries safe? Use explicit run IDs/idempotency keys, transactions, uniqueness rules and retry only stages whose effects are controlled.

50. Can prompt injection execute server code? No such execution tool is present, but injected document instructions can still manipulate answers. Keeping unauthorized data out of prompts is essential.

Testing, deployment and product

51. What did the review actually verify? Typecheck and production build passed; lint had six warnings; npm ci failed on lockfile consistency; workaround installation succeeded. Database/Gemini workflows were not run.

52. How would you measure accuracy? With a labeled source dataset and separate numeric/unit/period, risk and citation metrics. No measured accuracy figure exists in this snapshot.

53. Why might a free serverless host fail? Request limits and post-response lifecycle can stop unawaited document/report jobs. Pricing and platform promises must be verified rather than assumed.

54. Is report PDF export built in? No. App reports export TXT. A separate documentation script and this handbook produce PDFs.

55. Why does the demo use hard-coded financial data? It enables a populated educational UI without live extraction. It should be labeled fixture data and verified before being used as a factual financial source.

56. What happens if you register before seeding? seedDatabase detects an existing user and skips the whole seed, although the endpoint still reports success.

57. What would you improve first? Authorization, secret/seeding safety, lockfile reproducibility, runtime validation and unit correctness; then durable jobs and evidence retrieval.

58. What is the project's main engineering contribution? Integrating document ingestion, structured analysis and a research UI through clearly separated modules and persistent data. Do not present it as a new financial model or new LLM architecture.

59. Can it answer every financial question? No. Answers are limited by uploaded evidence, truncation, retrieval quality, model behavior and missing data. A good system should explicitly abstain when evidence is absent.

60. How should an uncertain answer be presented? State what is supported, cite the available evidence, identify missing period/unit/context and avoid turning absence into invented certainty or investment advice.

28. Glossary and quick revision sheet

Agent: here, a specialized TypeScript processing module. **API:** an interface for requests between software components. **App Router:** Next.js routing based on directories and special filenames. **Async:** work that completes later through a Promise. **Authentication:** confirming identity. **Authorization:** checking permission for a resource.

Benchmark: comparison against peers or a defined reference. **Chunk:** a text window used for retrieval. **Citation:** a pointer/excerpt intended to support a claim. **Client:** the browser-side application. **Connection pool:** reusable database connections. **Context window:** the model's finite input/output capacity.

Decimal ratio: 0.25 representing 25%. **Drizzle:** typed SQL/schema tooling. **Embedding:** a vector representation of meaning, not used in current retrieval. **Fallback:** reduced-capability alternate behavior after failure. **Foreign key:** a database relationship constraint. **Grounding:** conditioning output on supporting evidence.

Hallucination: plausible but unsupported model output. **Idempotency:** repeating an operation without unintended duplicate effects. **Inference:** using a trained model to generate output. **JSONB:** PostgreSQL's binary JSON storage type. **JWT:** a token format used by the configured session strategy. **Migration:** a versioned database-schema change.

N+1 queries: one collection query plus per-item queries. **OCR:** converting document images into text. **ORM:** an abstraction for working with relational data; here Drizzle is SQL-oriented. **Prompt:** model instructions plus context. **RAG:** retrieval-augmented generation. **Regex:** a text-matching pattern.

Seed: initial demonstration data. **Server:** backend runtime handling trusted operations. **Tenant isolation:** separating one user's/organization's data from another's. **Token:** a model text unit, not the same as a character. **Transaction:** atomic group of database operations. **Vector search:** similarity search over embeddings, not implemented here.

Remember these ten facts: six agent modules; one Next.js app; eleven SQL tables; 1,500/200-character chunking; keyword retrieval; 8,000-character metrics excerpt; 10,000-character risks excerpt; four maximum transient-call attempts; TXT report downloads; no claim of production security or verified financial accuracy.

29. Maintaining and regenerating this handbook

This handbook is intentionally separate from the older public documentation, preserving those files and the application's behavior. The editable source is docs/FINRESEARCH_AI_PROJECT_HANDBOOK.md. The PDF generator is scripts/generate-project-handbook.py. It uses Python ReportLab only for this documentation task; Python is not part of the application's runtime stack.

To rebuild the PDF in a Python virtual environment:

```
python3 -m venv .venv
.venv/bin/pip install -r docs/handbook-requirements.txt
.venv/bin/python scripts/generate-project-handbook.py
```

The generator requires DejaVu Sans, DejaVu Sans Bold and DejaVu Sans Mono fonts. It looks in /usr/share/fonts/truetype/dejavu on Linux; on another system, install the DejaVu font family and pass --font-dir with the directory containing those TTF files.

The generator creates a searchable PDF with a linked table of contents, headings, page numbers and wrapped code/reference tables. The reviewed database dictionary and source inventory below are derived from this snapshot and included in the Markdown; regenerate/update those explanations when code changes. The PDF renderer alone does not re-audit application behavior.

When maintaining documentation, update the source commit/date, re-run checks, verify model availability separately, inspect API scopes and re-check every implementation claim. Use source paths in this handbook to find the underlying evidence. The repository is the primary source; no external market numbers or provider pricing were needed to explain it.

The appendices that follow provide a field-by-field database dictionary, a complete tracked-file guide and a compact agent-function reference.

Appendix A. Complete database field dictionary

Generated against src/db/schema.ts in the reviewed snapshot. SQL column names are shown after their TypeScript property names. Unless marked required, a column permits NULL; a default does not itself imply NOT NULL. UUID defaults and serial keys are database-generated. Monetary/ratio meanings are intended conventions, not enforced unit checks.

users (users)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
email / email	varchar(255); required; unique	Normalized to lowercase by registration/login; unique in the database.
name / name	varchar(255); required	Human-readable display name; not a globally unique company identity.
password / password	text; required	bcrypt hash, not plaintext in normal registration/seed behavior.
role / role	varchar(50); required; default "analyst"	Role label carried in sessions; not by itself an enforced permission policy.
avatar / avatar	text; nullable	Optional avatar value; no avatar upload workflow was found.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.
updatedAt / updated_at	timestamp; required; now() default	Insertion default; later changes require explicit updates in application code.

research_sessions (researchSessions)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
userId / user_id	uuid; required; FK users.id; delete cascade	Owning/associated user reference; must still be checked in authorization queries.
name / name	varchar(255); required	Human-readable display name; not a globally unique company identity.
description / description	text; nullable	Optional descriptive text, except a risk description is required.
status / status	varchar(50); required; default "active"	Text lifecycle label, not a database-enforced enum.
tags / tags	text[]; nullable	Optional array of workspace labels.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.
updatedAt / updated_at	timestamp; required; now() default	Insertion default; later changes require explicit updates in application code.

companies (companies)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
name / name	varchar(255); required	Human-readable display name; not a globally unique company identity.
ticker / ticker	varchar(20); nullable	Optional stock-symbol label; not a live-price integration or unique key.
sector / sector	varchar(100); nullable	Optional high-level company sector.
industry / industry	varchar(100); nullable	Optional industry classification.
description / description	text; nullable	Optional descriptive text, except a risk description is required.
isSeeded / is_seeded	boolean; nullable; default false	Demo/seed visibility marker; nullable because notNull is not declared.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.

documents (documents)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
sessionId / session_id	uuid; nullable; FK researchSessions.id; delete cascade	Research workspace association. Log references are labels, not enforced foreign keys.
companyId / company_id	uuid; nullable; FK companies.id; delete set null	Company association used to aggregate financial values and risks.
userId / user_id	uuid; required; FK users.id; delete cascade	Owning/associated user reference; must still be checked in authorization queries.
fileName / file_name	varchar(500); required	Original upload name or generated seed filename.
fileType / file_type	varchar(50); required	Upload extension label; seeded rows use text/plain.
fileSize / file_size	integer; nullable	Uploaded byte size; seeded content uses string length.
documentType / document_type	varchar(100); required	User-selected category such as annual report; not validated as an enum.
fiscalYear / fiscal_year	integer; nullable	Period year; can be missing and can differ between document and extracted metric row.
content / content	text; nullable	Extracted raw document text, cleaned chunk text, or message text depending on table.
summary / summary	text; nullable	Optional document summary; seed populates it, upload analysis does not generate one.
chunkCount / chunk_count	integer; nullable; default 0	Stored chunk count; default zero, not declared notNull.
isSeeded / is_seeded	boolean; nullable; default false	Demo/seed visibility marker; nullable because notNull is not declared.
processingStatus / processing_status	varchar(50); required; default "pending"	Document workflow state: pending, processing, indexed, completed or failed by convention.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.
updatedAt / updated_at	timestamp; required; now() default	Insertion default; later changes require explicit updates in application code.

document_chunks (documentChunks)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
documentId / document_id	uuid; required; FK documents.id; delete cascade	Source document association. Log references are not enforced foreign keys.
chunkIndex / chunk_index	integer; required	Zero-based position of the text window; no declared document/index uniqueness.
content / content	text; required	Extracted raw document text, cleaned chunk text, or message text depending on table.
pageNumber / page_number	integer; nullable	Estimated as floor(index/3)+1, not an authentic PDF page.
section / section	varchar(255); nullable	Approximate heading label for a chunk.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.

financial_metrics (financialMetrics)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
documentId / document_id	uuid; required; FK documents.id; delete cascade	Source document association. Log references are not enforced foreign keys.
companyId / company_id	uuid; nullable; FK companies.id; delete cascade	Company association used to aggregate financial values and risks.
fiscalYear / fiscal_year	integer; nullable	Period year; can be missing and can differ between document and extracted metric row.
fiscalPeriod / fiscal_period	varchar(20); nullable	Annual/quarter label by prompt convention; local extraction forces Annual.
revenue / revenue	decimal(20,2); nullable	Revenue, intended USD millions.
revenueGrowth / revenue_growth	decimal(10,4); nullable	Revenue growth as a decimal fraction.
grossProfit / gross_profit	decimal(20,2); nullable	Gross profit, intended USD millions.
grossMargin / gross_margin	decimal(10,4); nullable	Gross profit/revenue as a decimal fraction.
operatingIncome / operating_income	decimal(20,2); nullable	Operating profit, intended USD millions.
operatingMargin / operating_margin	decimal(10,4); nullable	Operating income/revenue fraction.
netIncome / net_income	decimal(20,2); nullable	Net profit, intended USD millions.
netMargin / net_margin	decimal(10,4); nullable	Net income/revenue fraction.
ebitda / ebitda	decimal(20,2); nullable	EBITDA, intended USD millions; definition depends on source.
ebitdaMargin / ebitda_margin	decimal(10,4); nullable	EBITDA/revenue fraction.
totalAssets / total_assets	decimal(20,2); nullable	Balance-sheet assets, intended USD millions.
totalLiabilities / total_liabilities	decimal(20,2); nullable	All liabilities, intended USD millions.
totalEquity / total_equity	decimal(20,2); nullable	Balance-sheet equity, intended USD millions.
cashAndEquivalents / cash_and_equivalents	decimal(20,2); nullable	Cash/near-cash, intended USD millions.
totalDebt / total_debt	decimal(20,2); nullable	Borrowings, intended USD millions; not total liabilities.
currentRatio / current_ratio	decimal(10,4); nullable	Current assets/current liabilities, a multiple.

Property / SQL name	SQL type and rules	Meaning in this project
quickRatio / quick_ratio	decimal(10,4); nullable	Quick/liquid assets/current liabilities, source-defined multiple.
debtToEquity / debt_to_equity	decimal(10,4); nullable	Debt/equity multiple, sensitive to low or negative equity.
roe / roe	decimal(10,4); nullable	Return on equity as a decimal fraction; denominator definition requires review.
roa / roa	decimal(10,4); nullable	Return on assets as a decimal fraction.
eps / eps	decimal(10,4); nullable	Earnings per share in dollars, not millions; basis not stored separately.
peRatio / pe_ratio	decimal(10,4); nullable	Price/earnings multiple; no live price source.
operatingCashFlow / operating_cash_flow	decimal(20,2); nullable	Operating cash flow, intended USD millions.
capitalExpenditures / capital_expenditures	decimal(20,2); nullable	Capex, intended USD millions; sign convention must be checked.
freeCashFlow / free_cash_flow	decimal(20,2); nullable	Free cash flow, intended USD millions; commonly operating cash flow minus capex.
rawMetrics / raw_metrics	jsonb; nullable	Flexible metadata/additional values; local path marks extraction_method.
extractedAt / extracted_at	timestamp; required; now() default	Metric insertion timestamp, also used in latest-row sorting.

risk_flags (riskFlags)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
documentId / document_id	uuid; required; FK documents.id; delete cascade	Source document association. Log references are not enforced foreign keys.
companyId / company_id	uuid; nullable; FK companies.id; delete cascade	Company association used to aggregate financial values and risks.
riskType / risk_type	varchar(100); required	Risk-category label, not a database enum.
severity / severity	varchar(20); required	Critical/high/medium/low by convention; local fallback always medium.
title / title	varchar(500); required	Display title; for risk this is a short signal, for report/benchmark a name.
description / description	text; required	Optional descriptive text, except a risk description is required.
sourceText / source_text	text; nullable	Supporting quote candidate, not automatically validated against the source.
pageReference / page_reference	varchar(100); nullable	Optional risk reference; exact page support is not implemented.
recommendation / recommendation	text; nullable	Suggested action from model or local rule.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.

analysis_reports (analysisReports)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
sessionId / session_id	uuid; nullable; FK researchSessions.id; delete cascade	Research workspace association. Log references are labels, not enforced foreign keys.
userId / user_id	uuid; required; FK users.id; delete cascade	Owning/associated user reference; must still be checked in authorization queries.
title / title	varchar(500); required	Display title; for risk this is a short signal, for report/benchmark a name.
reportType / report_type	varchar(100); required	Stored report-category label; current generator does not branch on it.
companies / companies	text[]; nullable	Snapshot of company display names, not foreign-key IDs.
executiveSummary / executive_summary	text; nullable	Extracted first report section or first 500 characters.
keyFindings / key_findings	jsonb; nullable	Generator writes total-company/risk/critical-risk counts.
metricsComparison / metrics_comparison	jsonb; nullable	Schema field not populated by the current report generator.
riskSummary / risk_summary	jsonb; nullable	Generator stores risk counts by company.
recommendations / recommendations	text[]; nullable	Array of extracted or generic report recommendation strings.
fullReportContent / full_report_content	text; nullable	Full generated or local report text, used for TXT download.
agentLogs / agent_logs	jsonb; nullable	Optional embedded report metadata, not filled by current generator; distinct from log table.
status / status	varchar(50); required; default "generating"	Text lifecycle label, not a database-enforced enum.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.
updatedAt / updated_at	timestamp; required; now() default	Insertion default; later changes require explicit updates in application code.

chat_messages (chatMessages)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
sessionId / session_id	uuid; required; FK researchSessions.id; delete cascade	Research workspace association. Log references are labels, not enforced foreign keys.
userId / user_id	uuid; required; FK users.id; delete cascade	Owning/associated user reference; must still be checked in authorization queries.
role / role	varchar(20); required	Conversation speaker label, normally user or assistant; distinct from account role.
content / content	text; required	Extracted raw document text, cleaned chunk text, or message text depending on table.
citations / citations	jsonb; nullable	Array-like JSON of documentName, section and excerpt objects.
agentType / agent_type	varchar(100); nullable	Assistant/source label such as Research Agent, Demo or Error.
reasoning / reasoning	text; nullable	Application-generated evidence-count summary; not an audited model reasoning trace.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.

benchmark_comparisons (benchmarkComparisons)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	uuid; primary key; random UUID default	Primary identifier; generated UUID except the serial log ID.
sessionId / session_id	uuid; nullable; FK researchSessions.id; delete cascade	Research workspace association. Log references are labels, not enforced foreign keys.
userId / user_id	uuid; required; FK users.id; delete cascade	Owning/associated user reference; must still be checked in authorization queries.
title / title	varchar(500); required	Display title; for risk this is a short signal, for report/benchmark a name.
companyIds / company_ids	text[]; nullable	Text array planned for saved benchmarks; not FK-enforced and route does not insert it.
metrics / metrics	jsonb; nullable	Flexible saved comparison payload; current route does not persist comparisons.
insights / insights	text; nullable	Saved comparison prose provision; not written by current route.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.

agent_logs (agentLogs)

Property / SQL name	SQL type and rules	Meaning in this project
id / id	serial; primary key	Primary identifier; generated UUID except the serial log ID.
sessionId / session_id	uuid; nullable	Research workspace association. Log references are labels, not enforced foreign keys.
documentId / document_id	uuid; nullable	Source document association. Log references are not enforced foreign keys.
agentName / agent_name	varchar(100); required	Human-readable processing-module name.
action / action	varchar(255); required	Human-readable event description.
status / status	varchar(50); required	Text lifecycle label, not a database-enforced enum.
details / details	text; nullable	Diagnostic context; may contain questions/errors and needs scoped access/retention.
duration / duration	integer; nullable	Milliseconds elapsed for a logged agent event.
createdAt / created_at	timestamp; required; now() default	Insertion timestamp default; no timezone annotation in schema.

Appendix B. Every tracked file and where to start

The reviewed baseline contains 69 tracked files, including 50 application files under src. This inventory excludes .git internals, installed dependencies, build output and the new handbook deliverables. Read in this order for fastest understanding: package.json -> src/app/page.tsx -> src/lib/auth.ts -> src/db/schema.ts -> documents API -> agent modules -> related dashboard pages.

.env.example

Template for database/auth/provider environment variables. Its model IDs differ from source defaults; configure only verified provider models. No actual secret is supplied.

.gitignore

Excludes node_modules, Next build output, common environment files, logs and credential patterns. It does not make already-tracked secrets safe; no live credential audit is implied.

README.md

Existing overview, setup and feature narrative. Useful orientation, but some claims such as five agents do not match the six agent modules.

drizzle.config.json

Drizzle Kit configuration selecting PostgreSQL and src/db/schema.ts, with a hard-coded local connection URL. Runtime db/index.ts instead reads DATABASE_URL.

drizzle/0000_bizarre_enchantress.sql

Initial migration: creates all 11 tables, primary/unique constraints and foreign keys. No explicit secondary performance indexes or tenant-security policies.

drizzle/meta/0000_snapshot.json

Machine-generated Drizzle snapshot of the initial schema. Tooling metadata, not application business logic or a database backup.

drizzle/meta/_journal.json

Migration journal metadata for the initial migration tag and breakpoint handling. Do not confuse it with an application activity log.

eslint.config.mjs

Flat ESLint configuration using Next core-web-vitals rules and ignoring generated output. Typecheck is a separate npm script.

next-env.d.ts

Next-generated type references used by TypeScript. Normally maintained by tooling rather than manual feature edits.

next.config.ts

Minimal Next configuration containing turbopack.root. Actual dev/build commands choose webpack explicitly.

package-lock.json

Resolved dependency graph for reproducible installation in principle. This review found a picomatch consistency problem blocking npm ci.

package.json

Project metadata, five npm scripts and runtime/development dependency declarations. The package name remains nextjs-postgresql-template.

postcss.config.mjs

PostCSS plugin configuration integrating Tailwind v4 into the CSS build.

public/DOCUMENTATION.md

Older publicly served Markdown guide; the dashboard Docs download points here.

public/FINRESEARCH_AI_COMPLETE_DOCUMENTATION.md

Existing long-form documentation source. Preserved; this handbook adds a new source-verified explanation rather than overwriting it.

public/FINRESEARCH_AI_COMPLETE_DOCUMENTATION.pdf

Existing generated PDF associated with the older source. It is distinct from the new reviewed handbook.

scripts/generate-documentation-pdf.cjs

Original Node/jsPDF Markdown renderer. Hard-coded input/output public filenames; basic headings/code/bullets/tables and footers. Table layout assumes three columns, and no generated linked contents exists.

src/app/api/auth/[...nextauth]/route.ts

Creates the NextAuth handler and exports it for GET/POST.

src/app/api/auth/register/route.ts

Registration database write: presence/duplicate checks, lowercase email, bcrypt cost 10, safe selected response fields.

src/app/api/benchmark/route.ts

On-demand comparison response; owner-or-seeded filters on metric/risk document joins. Does not persist benchmark rows.

src/app/api/chat/route.ts

Chat history and message persistence. Branches to hard-coded no-key demo or Research Agent. Missing supplied-session ownership check.

src/app/api/companies/route.ts

Seeded-or-user-associated company list; per-company metric/risk/document queries are not separately owner-scoped.

src/app/api/dashboard/route.ts

Mix of user-scoped personal counts/lists and global company/risk/log/metric data. Global parts require isolation fixes.

src/app/api/documents/[id]/route.ts

Owner-scoped document read with associated metrics/risks/chunk metadata and owner-scoped delete.

src/app/api/documents/route.ts

Owner-scoped document list and multipart upload parser. Starts the sequential three-stage pipeline without awaiting completion.

src/app/api/health/route.ts

Public SELECT 1 liveness/database probe; returns static agent list and app version, raw error on failure.

src/app/api/metrics/route.ts

Filtered or global financial metrics query after login, missing tenant/document ownership enforcement.

src/app/api/reports/[id]/route.ts

Owner-scoped report get/delete.

src/app/api/reports/route.ts

Owned report list and asynchronous report creation. Validates a collective available-document condition, not each selected company or optional session ownership. Contains an unused generateDemoReport function.

src/app/api/risks/route.ts

Filtered or global risk query after login, missing ownership filtering; severity input is unused.

src/app/api/seed/route.ts

Public GET/POST seed invocation. Should be disabled/guarded outside disposable demos.

src/app/api/sessions/[id]/route.ts

Owner-scoped get/patch/delete with an explicit update-field allowlist.

src/app/api/sessions/route.ts

Owned session list with per-session counts and session creation.

src/app/dashboard/benchmark/page.tsx

Company selection, comparison POST and charts/table/insights. Selects first matching metric row without explicit period selection.

src/app/dashboard/companies/page.tsx

Company cards and summary figures with navigation to metrics, risks and benchmark views.

src/app/dashboard/docs/page.tsx

Static in-app documentation with tab selection and a Markdown download action. Not automatically derived from code.

src/app/dashboard/documents/[id]/page.tsx

Owned document details with overview/metrics/risks/content tabs. Displays extracted text, not original binary document rendering.

src/app/dashboard/documents/page.tsx

Single-file upload dropzone and optional company/session/year form, document search/list/delete and one delayed status refresh.

src/app/dashboard/layout.tsx

Session-aware dashboard shell, navigation links, responsive sidebar, active-route styles and sign-out. Client redirect is not server authorization.

src/app/dashboard/metrics/page.tsx

Financial table plus Recharts bar/radar views. Converts stored values for display; null-to-zero and chart-unit issues require care.

src/app/dashboard/page.tsx

Dashboard data fetch, aggregate cards, recent activity and demo-seed action. Some returned aggregates are global in the API.

src/app/dashboard/reports/[id]/page.tsx

Report detail, manual refresh while generating, simple line-based formatted rendering and TXT export. Unused renderMarkdown helper is not active HTML injection.

src/app/dashboard/reports/page.tsx

Report collection, generation form, company/session selection, delete and client TXT export.

src/app/dashboard/research/page.tsx

Session-based chat with optimistic messages, suggestion handling, loading, auto-scroll and expandable citations. No streaming.

src/app/dashboard/risks/page.tsx

Risk cards with local severity/type filters and severity counts. Uses predefined critical/high/medium/low sort order.

src/app/dashboard/sessions/[id]/page.tsx

Session details plus document list loaded in parallel. Navigation query parameters are not consistently consumed by destination pages.

src/app/dashboard/sessions/page.tsx

Session list/search and create/edit/delete form. Converts tag input to an array and refreshes after mutations.

src/app/globals.css

Tailwind import, system fonts, scrollbar styles, report typography, keyframe animations and chat-bubble styling.

src/app/layout.tsx

Global document shell, title/description metadata and provider wrapper. References /favicon.ico, but that asset is not in the reviewed tracked file inventory.

src/app/login/page.tsx

Email/password sign-in UI using NextAuth credentials; includes demo fill and dashboard navigation.

src/app/page.tsx

Async server entry route; checks session and redirects to /dashboard or /login.

src/app/providers.tsx

Client SessionProvider plus configured top-right toast notifications. Makes client authentication state available to child pages.

src/app/register/page.tsx

Registration UI with name/email/password fields, eight-character client check and fetch to the register API.

src/db/index.ts

Constructs a pg Pool from DATABASE_URL and initializes Drizzle with the schema. No custom pool sizing/timeouts.

src/db/schema.ts

Definitions for 11 tables, selected Drizzle relations and exported inferred TypeScript types. Database field dictionary above covers each column.

src/lib/agents/benchmarkAgent.ts

Latest-intended company metric profiles, AI comparison, ROE-ranked local fallback and activity logging.

src/lib/agents/documentAgent.ts

Text cleaning/chunking/section detection, database chunk writes and simple keyword search. No embeddings or OCR.

src/lib/agents/extractionAgent.ts

8,000-character Gemini metric extraction, mapping/insertion and local regex fallback with unit limitations.

src/lib/agents/gemini.ts

Lazy client, model environment defaults, transient retry wrapper, free-text and JSON generation helpers.

src/lib/agents/reportAgent.ts

Company-based report prompt, local template, summary/recommendation regex extraction and report status updates.

src/lib/agents/researchAgent.ts

Session evidence gathering, prompt/history assembly, attached citations and local evidence summary. userId argument does not constrain queries.

src/lib/agents/riskAgent.ts

10,000-character AI risk prompt plus five local keyword-check categories and risk persistence.

src/lib/auth.ts

Credentials provider, bcrypt comparison, JWT/session callbacks, 30-day sessions and login page configuration. Contains unsafe predictable secret fallback.

src/lib/seed.ts

Creates two demo users, four companies, one main research session, hard-coded text/metrics/risks and example logs. Skips everything if any user exists.

src/lib/seedData.ts

Hard-coded Apple/Microsoft/Tesla/Amazon fixtures. Educational input rather than verified downloaded filings; contains internal data inconsistencies.

src/lib/utils.ts

Class merging, currency/percent/number/date formatting, initials, truncation and severity/status CSS mappings.

src/types/next-auth.d.ts

Module augmentation giving User/Session/JWT their project-specific ID/role type fields.

tsconfig.json

Strict TypeScript config, ES2019 target, bundler module resolution, JSX support, Next plugin and @/src alias mapping.

tsconfig.tsbuildinfo

Tracked incremental compiler cache. Generated tooling state, not business logic; can change on typecheck/build.

Appendix C. Agent function and data-contract map

This compact map connects names you will see in the code to their caller, output and principal side effect. Helper names that are not exported are implementation details, not API endpoints.

chunkText(text, chunkSize=1500, overlap=200)

Called by processDocument; returns string[]. Pure slicing loop with no DB or AI call. Validate overlap if exposing arbitrary options.

extractSections(text)

Called by processDocument; returns section/content objects by recognized heading patterns. It does not infer a document outline with an LLM.

processDocument(documentId, content)

Called by upload pipeline and seeding; returns Promise<void>. Writes log events/chunks and document state/count.

searchDocumentChunks(documentId, query, limit=5)

Called by both research modes; returns content/section/chunkIndex candidates. Loads up to 100 database chunks and scores keywords locally.

findNumber(content, labels)

Private extraction helper. Returns first matching number or undefined; only billion gets monetary scaling and percent is not normalized.

extractMetricsLocally(content)

Private fallback helper. Returns partial ExtractedMetrics and extraction_method marker; no independent AI request.

extractFinancialMetrics(documentId, content, companyId?)

Upload stage; returns Promise<void>. Uses generateJSON, inserts metric rows, or performs local fallback; writes activity.

findLocalRisks(content)

Private risk helper. Returns zero to five medium-severity indicators from keyword categories.

scanForRisks(documentId, content, companyId?)

Upload stage; returns Promise<void>. Inserts AI or rule-derived risk rows and logs.

getGeminiClient()

Returns cached SDK client; throws for missing/demo key. Server-side configuration only.

generateContentWithRetry(model, prompt)

Private SDK wrapper. Retries selected transient error strings with exponential waits, then returns response or throws.

generateWithGemini(prompt, systemInstruction?, modelName?)

Returns Promise<string> for research/benchmark/report prose. No runtime financial validation.

generateJSON<T>(prompt, systemInstruction, modelName?)

Returns parsed model JSON asserted as T. T is not a runtime validator.

buildLocalResearchAnswer(sessionId, question)

Private research fallback. Returns answer, citation objects and a retrieval-count summary.

answerResearchQuestion(sessionId, userId, question, conversationHistory)

Called by chat when a non-demo key exists. Returns answer/citations/reasoning; chat route persists messages. User ID is not used for query scoping inside agent.

generateBenchmarkInsights(companyIds, userId, sessionId?)

Called by benchmark route when a key exists. Returns narrative string, using permitted document rows; does not save benchmark_comparisons.

createLocalBenchmarkInsights(context)

Private ROE ranking formatter. Missing ROE becomes zero for sorting; returns a warning-labeled narrative.

generateAnalysisReport(reportId, sessionId, userId, companyIds, additionalContext?)

Called unawaited from report creation. Returns Promise<void>; updates stored report and logs; filters permitted documents but not session-only documents.

createLocalReport(profiles, reason)

Private report fallback formatter. Includes extracted values, risks, warnings and raw provider reason; should sanitize provider detail in production.

Appendix D. Source evidence and review navigation

All implementation statements in this handbook are based on the reviewed repository, not live market information. The authoritative source groups below provide an audit trail without fragile line-number references. Paths and named functions remain useful when formatting changes shift line numbers.

Architecture and ingestion: `src/app/api/documents/route.ts` and `src/lib/agents/documentAgent.ts`. Check `POST` and `processDocument` for the exact stage order and status transitions.

Financial/risk semantics: `src/lib/agents/extractionAgent.ts`, `riskAgent.ts` and `src/lib/utils.ts`. Compare `EXTRACTION_SYSTEM` with `findNumber` and `formatPercent` to see why the fallback unit mismatch matters.

Research grounding and modes: `src/app/api/chat/route.ts` plus `researchAgent.ts` and `searchDocumentChunks`. Check `hasAI` before assuming a query reached retrieval.

Comparisons/reports: `benchmark/report` routes and agent files plus matching dashboard pages. Compare SQL ordering with UI find calls and inspect Blob download extensions.

Security and tenancy: `auth.ts`; every route where clause; Research Agent queries; `seed.ts`; dashboard aggregate queries. Login checks and ownership checks must be inspected separately.

Schema and deployment: `db/schema.ts`, `db/index.ts`, initial SQL migration, `package.json`, `package-lock.json`, `.env.example`, `drizzle.config.json` and `next.config.ts`. Environment defaults, migration targets and installed dependencies are separate contracts.

UI reality: all dashboard page files and `globals.css`. These are better evidence than old screenshots or marketing prose for filtering, navigation, rendering and refresh behavior.

Validation performed: typecheck, lint and webpack build outputs were examined during this review. No claim is made about provider availability, live accuracy, private-data isolation tests or production readiness beyond the explicit checks in Chapter 21.

End of handbook. The strongest way to know this project is to trace a request, inspect the stored evidence, reproduce a small example and explain both the successful path and its failure modes.